Gimel Foundation

Gimel Foundation gGmbH i.G.
GiFo-Request for Comments: 0150
Obsoletes: -
Category: Standards Track
Version: 0.9.5 (Draft – Under Vendor Review)

G. Wehberg, P. Baran and the
Architecture Working Group of
Gimel Foundation
20. April 2026

G-PaC Integration Profile - Combined Credential- and Platform-bound Enforcement (CCPE-B)

Abstract

A second integration pattern within the “Combined Credential- and Platform-Bound Enforcement (CCPE)” family is now sufficiently visible to specify. Where GiFo-RFC 0140 (G-AGT Integration Profile) defines how GAuth's credential-bound enforcement integrates with “enterprise governance toolkits” (AGT-class engines, designated CCPE-A), this RFC defines how GAuth integrates with standalone Policy-as-Code (PaC) engines - Open Policy Agent (OPA / Rego), Cedar (AWS), and Zanzibar / SpiceDB (Authzed) - designated CCPE-B.

The two profiles share a common credential layer (GAuth's Power-Enforcement Point and the 16-check pipeline of GiFo-RFC 0117) and differ in the architectural shape of the platform-bound Phase 2 evaluator they integrate with. CCPE-A and CCPE-B are not alternatives in the sense that one displaces the other; they address different deployment realities - organizations whose authorization story is anchored in an AGT-class governance toolkit (CCPE-A) versus organizations whose authorization story is anchored in an existing PaC deployment with its own bundle servers, policy administrators, and pre-existing application embedments (CCPE-B).

This RFC is the engine-neutral integration profile. Its purpose is to specify how GAuth's Power-PEP chains to a downstream Policy-as-Code Policy-PDP, what must be true of the PaC engine for the integration to be conformant, and what the credential layer (and optionally the Gimel Auth Platform as the proprietary value-add layer) must supply that the PaC engine does not. Per-vendor cookbooks are explicitly out of scope and are deferred to companion documents or vendor-led contributions.

This document is published as a v0.9 draft under vendor review. Standards-body amplification to OPA / CNCF, AWS Cedar working group, and Authzed is sequenced after RFC 0140 v1.2 (which introduces the CCPE term and family taxonomy) has had its initial circulation period. The publication date itself is the relevant artifact — it establishes the credential-bound + Policy-as-Code integration architecture specification.

Status of This Memo — Work-in-Progress / Under Vendor Review

This document is published as a work-in-progress draft and is currently under vendor review. Engine descriptions are sourced from the engines' own published documentation (OPA/CNCF, AWS Cedar, Authzed/SpiceDB) and from the contributor-implementation work surveyed in §3. No engine characterization is original analysis without an attributed source. The integration architecture (§4–§7) is the original value-add of this document; the engine descriptions are aggregated public material.

Corrections, clarifications, and additions from the engine maintainers and contributor implementations named in §3 are actively solicited and will be incorporated into subsequent revisions. A correction channel is described in §10.

Evaluative claims in §7 are back-cited in Appendix A (per-engine citation ledger), which carries claim → source URL → access date → verbatim evidence rows for each in-scope engine. Engine maintainers are invited to challenge any row through §10. Where this draft uses qualitative language about a category, the language refers to the architectural pattern as documented in publicly available materials at the date of drafting; specific engine implementations may differ. The integration profile in §4–§6 is intended to be reusable by any party - including the named engine maintainers - who wishes to apply it to their own architecture and publish a counter-evaluation.

This is a Gimel Foundation Standards Track document. It represents the current consensus of the Gimel Foundation community pending the engine-maintainer review cycle. Information about the status of this document, any errata, and how to provide feedback may be obtained at https://gimelfoundation.com or https://github.com/Gimel-Foundation.

Legal Notice

Copyright (c) 2026 Gimel Foundation and the persons identified as the document authors. All rights are reserved.

This document is subject to GiFo-RFC 0080 (Gimel Foundation Legal Provisions Relating to GiFo Documents), GiFo-RFC 0090 (Gimel Foundation Rights in Contributions), and GiFo-RFC 0100 (Gimel Foundation Intellectual Property Rights Policy) in effect on the date of publication of this document (see http://GimelFoundation.com or https://github.com/Gimel-Foundation). Please review these documents carefully, as they describe your rights and restrictions with respect to this document.

This specification is published under the Apache License 2.0, consistent with GiFo-RFCs 0110, 0111, 0115, 0116, 0117, 0118, 0130, and 0140. Legal terms of Gimel Foundation apply.

Trademark Notice. Product names, trademarks, and registered trademarks referenced in this document are the property of their respective owners and are used here for descriptive identification purposes only under nominative fair use principles. No endorsement, sponsorship, partnership, or affiliation is implied. Open Policy Agent (OPA), Rego, Cedar, AWS, Zanzibar, SpiceDB, Authzed, Permit.io, Styra, OpenFGA, Microsoft, AGT, and the contributor implementations named in §3 are referenced descriptively as exemplar Policy-as-Code engines or contributor implementations of the architectural pattern defined herein. Each is governed by its own license and its own community-stewardship arrangements; this document does not modify or supersede any of them.

OWASP Notice. Where this document references the OWASP AI Security Solutions Landscape (inherited via cross-reference to GiFo-RFC 0130 §4), the OWASP non-endorsement disclaimer applies: OWASP states the Solutions Landscape is "not a comprehensive list or an endorsement". Inclusion in this document of any engine or contributor implementation reflects the engine's own published descriptions, not endorsement by the Gimel Foundation, the document authors, or OWASP.

Notational Conventions

The key words "Must", "Must Not", "Required", "Shall", "Shall Not", "Should", "Should Not", "Recommended", "May", and "Optional" in the following specification are to be interpreted as described in IETF RFC 2119.

Note on terminology - Power*Point vs. Policy*Point. GiFo-RFCs 0110 and 0111 normatively define “P*P” in the GAuth context as “Power-Decision / Enforcement / Administration / Information / Verification Point”, emphasizing that GAuth governs delegated *authority* (power), not platform-configured *policy*. This is intentionally distinct from the XACML / IETF RFC 2753 sense of P*P as “Policy*Point” (Policy-Decision / Enforcement / Administration / Information Point). The architectural decomposition is structurally analogous, but the bound artifact (a Power-of-Attorney credential vs. a policy document) is categorically different. This RFC is the natural home for that distinction: it specifies the chain in which GAuth's “Power-PEP” invokes a downstream “Policy-PDP” provided by a Policy-as-Code engine. Where this document uses the unqualified term “PEP” without a Power- / Policy- prefix, the prefix is determined by context: GAuth components are Power-* by definition (per RFC 0110/0111); PaC-engine components are Policy-* by definition (per XACML / IETF RFC 2753). Where ambiguity is possible, this document uses the prefixed forms.

Throughout this document, the term “CCPE” (Combined Credential- and Platform-Bound Enforcement) refers to the architectural family normatively introduced in GiFo-RFC 0140 v1.2. CCPE-A designates the integration with enterprise governance toolkits (RFC 0140). CCPE-B designates the integration with standalone Policy-as-Code engines (this RFC). CCPE-C is reserved for future engine families (e.g., research-tooling integration, hardware-attested enforcement, sector-specific engines) and is not specified herein.

* * *

Table of Contents

1. Scope

2. Terminology

3. Background: The Policy-as-Code Landscape for Agent Authorization

- 3.1 Engine Families in Scope
- 3.2 Contributor Implementations for Agent Hooks
- 3.3 Why a Standalone Spec

4. Integration Architecture

- 4.1 The Power-PEP → Policy-PDP Chain
- 4.2 PEP Placement Topologies
- 4.3 Decision-Shape Adapters
- 4.4 Adapter Type Reuse from RFC 0140

5. Trust-State Externalization and CCPE-B Engine Conformance

- 5.1 What PaC Engines Provide Natively
- 5.2 What PaC Engines Do Not Provide Natively
- 5.3 Trust-State PIP Contract
- 5.4 CCPE-B Engine Conformance Criteria

6. Multi-Tenancy and Policy-Isolation Models

- 6.1 OPA: Bundle-per-Tenant
- 6.2 Cedar: Policy-Store-per-Tenant
- 6.3 SpiceDB: Relationship-Namespace-per-Tenant
- 6.4 GAuth Credential-Native Multi-Tenancy
- 6.5 The Chaining Pattern

7. Comparative Analysis: CCPE-A vs. CCPE-B

8. Security Considerations

9. References

10. How to Suggest a Correction

11. Acknowledgments

1. Scope

1.1 What This RFC Does

This RFC:

a. **Defines the CCPE-B integration profile** within the Combined Credential- and Platform-Bound Enforcement family introduced normatively by GiFo-RFC 0140 v1.2.

b. **Specifies the Power-PEP → Policy-PDP chain** by which GAuth's credential-bound enforcement (Phase 1, per RFC 0140 §2.8) invokes a downstream Policy-as-Code engine as the platform-bound Phase 2 evaluator.

c. **Specifies the four architectural items** that distinguish CCPE-B from CCPE-A and that are not adequately specified by RFC 0140 alone: (i) PEP placement topology (centralized hook vs. application-embedded SDK), (ii) decision-shape normalization across heterogeneous engine return types, (iii) trust-state externalization as PIP inputs to engines that lack native trust scoring, and (iv) the explicit Power*Point ↔ Policy*Point chain semantics.

d. **Establishes engine-conformance criteria** appropriate to PaC engines (deterministic evaluation, multi-tenant policy isolation, structured decision return, ability to consume external PIP-fed trust state) — distinct from the AGT-class conformance criteria of RFC 0140 §4.3.

e. **Surfaces the boundary** between "AGT-class engine configured with Rego or Cedar syntax" (RFC 0140 / CCPE-A territory) and "standalone PaC engine deployment with its own bundle server, policy administrators, and pre-existing application embedments" (this RFC / CCPE-B territory).

1.2 What This RFC Does Not Do

This RFC is *not*:

- **A per-engine implementation guide in the normative core**. Engine-specific cookbook material is provided in informative Appendices B–D (OPA / Rego, Cedar, SpiceDB) for adopter convenience; those appendices are non-normative and may be superseded by engine-maintainer pull requests without revising the normative spec. Commercial-layer cookbooks (e.g., Styra DAS, Permit.io control plane, Casbin Cloud) are added incrementally as vendor reviewers confirm accuracy through the §10 channel.

- **A normative specification of the CCPE term itself**. The CCPE acronym, family taxonomy (CCPE-A / CCPE-B / CCPE-C reserved), and parent-level architectural framing are normatively introduced in GiFo-RFC 0140 v1.2. This RFC uses CCPE-B as already-defined and refers to RFC 0140 for the term's authoritative definition.

- **A modification to existing GiFo RFCs**. This document references published GiFo-RFCs 0080, 0090, 0100, 0110, 0111, 0115, 0116, 0117, 0118, 0130, and 0140; it does not modify any of them.

- **An evaluation of any specific PaC engine against the GiFo-RFC 0130 nine-criterion framework**. Engine maintainers are invited to publish such evaluations themselves using the framework as published.

- **A buyer's guide or product comparison**. The integration profile is descriptive of architectural integration shape, not prescriptive of engine selection.

- **A specification of Gimel Auth Platform proprietary services**. Where this document references the platform as the source of trust-state PIP feeds (§5.3), threat-detection corpus exposure, or managed adapter endpoints, those references describe the open interface the platform implements; the proprietary implementations are subject to separate licensing per the Type C adapter pattern of RFC 0140 §8.

1.3 Relationship to Other Specifications

- GiFo-RFC 0080 | Legal Provisions — governs the legal status of this document.
- GiFo-RFC 0090 | Rights in Contributions — governs engine-maintainer contributions to subsequent revisions.
- GiFo-RFC 0100 | Intellectual Property Rights Policy.
- GiFo-RFC 0101 *(in preparation)* | Patent Policy — once published, will govern the open-core patent license under which this specification is released. Until then, patent terms default to the Apache License 2.0 grant.
- GiFo-RFC 0110 | GAuth Protocol Engine — Power*Point architectural pattern; authoritative for the Power-* terminology used herein.
- GiFo-RFC 0111 | GAuth Authorization Framework — Power*Point semantics; reinforces Power-* usage.
- GiFo-RFC 0115 | Power-of-Attorney Credential Definition — the credential the Power-PEP enforces.
- GiFo-RFC 0116 | GAuth Interoperability — Extended Token (JWT with PoA claims), W3C VC representation.
- GiFo-RFC 0117 | GAuth PEP Interface — the 16-check pipeline that runs Phase 1 and invokes Phase 2 via the extension point this RFC integrates into.
- GiFo-RFC 0118 | GAuth Management API — mandate lifecycle, budget, governance profiles.
- GiFo-RFC 0130 | AI Governance Tool Evaluation — the PBRG category definition this RFC's "platform-bound" terminology inherits; nine-criterion framework reused in §7.

- GiFo-RFC 0140 - G-AGT Integration Profile (CCPE-A) — the sibling integration profile. Normatively introduces CCPE, Phase 1 / Phase 2 nomenclature, governance profiles, and the Type A/B/C adapter pattern this RFC builds on.
- IETF RFC 2119 | Notational conventions.
- IETF RFC 2753 | Framework for Policy-based Admission Control — origin of Policy*Point (XACML lineage).
- OASIS XACML 3.0 | Policy*Point decomposition for access control; origin terminology distinguished from Power*Point in the Notational Conventions note.

2. Terminology

This document uses the terminology established normatively in GiFo-RFCs 0110, 0111, 0117, 0130 §2, and 0140 §2. Where a term is defined in those documents, the definition there is authoritative. Terms specific to this RFC are defined below.

Policy-as-Code (PaC) Engine. A general-purpose authorization engine in which policy is expressed as machine-evaluable code (typically a domain-specific policy language) and evaluated against an input context to produce a structured decision. The exemplar engine families in scope of this RFC are Open Policy Agent (OPA, with the Rego policy language), Cedar (with the Cedar policy language), and Zanzibar / SpiceDB (with relationship tuples and a schema). Each is general-purpose by design (not AI-agent-specific) and predates the agent-governance use case.

Power*Point. GAuth-context shorthand for the five-component decomposition normatively defined in GiFo-RFC 0110 (and 0111, respectively): Power-Enforcement Point (Power-PEP), Power-Decision Point (Power-PDP), Power-Administration Point (Power-PAP), Power-Information Point (Power-PIP), and Power-Verification Point (Power-PVP). The bound artifact is a Power-of-Attorney (PoA) credential; the governed object is delegated authority. Distinct from Policy*Point.

Policy*Point. XACML / IETF RFC 2753 shorthand for the decomposition Policy-Enforcement Point, Policy-Decision Point, Policy-Administration Point, Policy-Information Point. The bound artifact is a policy document; the governed object is platform-configured policy. Distinct from Power*Point. Where this RFC integrates a PaC engine as the Phase 2 evaluator, the PaC engine's components are Policy-* by definition.

CCPE-B. The integration profile defined by this RFC: GAuth's Power-PEP (Phase 1, RFC 0117) chained to a standalone Policy-as-Code engine providing the platform-bound Policy-PDP and Policy-PAP (Phase 2). The "B" position is one of the CCPE family variants normatively introduced by RFC 0140 v1.2; CCPE-A (RFC 0140) and CCPE-B (this RFC) share the credential layer and differ in the Phase 2 engine pattern. CCPE-C is reserved for future engine families.

Phase 2 PaC Adapter. The architectural component that translates a Power-PEP decision request into the input shape expected by the PaC engine's Policy-PDP, invokes the engine, and normalizes the engine's structured decision back into the Power-PEP's decision envelope. The adapter is not an engine; it is the contract surface specified in §4.3. It corresponds, in the RFC 0140 §4.3 vocabulary, to the access-control engine adapter — re-specified here for engines that do not satisfy the AGT-class conformance criteria natively.

Centralized PEP Topology. A deployment topology in which a single Policy-Enforcement Point intercepts agent actions on behalf of multiple downstream services or tools. Microsoft AGT's `PreToolUse` hook is the exemplar centralized topology: one hook per agent runtime, all tool calls flow through it. CCPE-A operates in this topology by default.

Embedded PEP Topology. A deployment topology in which Policy-Enforcement Point invocation is embedded as SDK calls inside each application or service that participates in authorization. OPA, Cedar, and SpiceDB are typically deployed in this topology — every microservice that calls the engine is itself a Policy-PEP. The chaining pattern in §4.2 specifies how GAuth's centralized Power-PEP integrates with embedded Policy-PEP deployments without requiring rearchitecture of the existing PaC deployment.

Trust State. The collection of behavioral, deterministic-rule, and operational signals that GAuth (and/or the Gimel Auth Platform) maintains per agent and per credential — including but not limited to: trust score (where applicable), privilege ring assignment (where applicable), kill-switch state, circuit-breaker state, mandate status, and budget remaining. PaC engines do not maintain trust state natively; §5.3 specifies the PIP contract by which trust state is supplied to PaC engines as evaluation input.

Decision Envelope. The structured form a Phase 2 evaluator's decision must take to be consumable by GAuth's Power-PEP at the Phase 2 extension point. The minimum envelope is a triple `{verdict, obligations, diagnostics}` where `verdict ∈ {permit, deny}` and `obligations` carries optional structured constraints the Power-PEP applies before issuing its final PERMIT / DENY / CONSTRAIN. Heterogeneous PaC return shapes (Rego arbitrary structured results, Cedar Allow/Deny + diagnostics, SpiceDB boolean check + permission expansion) are normalized into the decision envelope by the Phase 2 PaC Adapter (§4.3).

3. Background: The Policy-as-Code Landscape for Agent Authorization

3.1 Engine Families in Scope

The Policy-as-Code engines in scope of CCPE-B are general-purpose authorization engines whose policy languages and evaluation models predate the AI-agent-governance use case. They share three architectural properties relevant to integration:

- **Decoupled policy and code**. Policy is expressed in a dedicated policy language (Rego, Cedar) or a relationship-and-schema model (SpiceDB), separately from the application code that invokes the engine.

- **Deterministic evaluation**. Given the same inputs, the engine returns the same decision. (AI-enhanced extensions of these engines exist; they are not in CCPE-B scope and are governed by the proprietary licensing terms of the engines that offer them.)

- **General-purpose by design**. None of the engines was designed for AI-agent governance specifically; each was designed for application-, microservice-, or relationship-level authorization. Use for agent-action interception is a retrofit.

The exemplar families and their published self-descriptions:

```
| Engine | Maintainer / Steward | Policy model | Typical
deployment |

|---|---|---|---|

| **Open Policy Agent (OPA)** with **Rego** | CNCF (graduated
project) | Declarative rules language (Rego); arbitrary
structured input/output | Sidecar, library, daemon; bundle
server distributes policy |

| **Cedar** | AWS (open source under Apache 2.0); also
independently maintained as Open Source Cedar | Strongly-
typed, schema-validated policy language; Allow/Deny with
diagnostics | Library-embedded; policy-store deployment |

| **Zanzibar / SpiceDB** | Authzed (open-source SpiceDB);
Google-pioneered Zanzibar pattern | Relationship tuples +
schema; permission checks by relationship traversal |
Networked authorization service; relationship database |
```

Adjacent engines and platforms that sit in or near this space — Permit.io, Styra (commercial OPA-based), OpenFGA (CNCF, Zanzibar-pattern), Casbin — share enough architectural shape to apply CCPE-B integration to. They are not separately enumerated below; engine-maintainer additions via §10 are welcome.

3.2 Contributor Implementations for Agent Hooks

The earliest reference implementations of PaC-engines-driving-agent-hooks were surveyed in GiFo-RFC 0130 §4.5 as the "Cedar/OPA-Based Policy Frameworks" related sub-category. They are reproduced here as the prior contributor surface this RFC builds on:

- **Sondera** — "Hooking Coding Agents with the Cedar Policy Language" (Matt Maisel; published blog post and conference talk). Demonstrates Cedar policies driving Claude Code hook decisions.

- **Cupcake** — EQTY Lab, with Trail of Bits. An OPA / Rego-based policy framework for agent actions.

- **OpenClaw** — open-source policy-aware agent loop with Cedar integration.

- **Phil Windley** — "A Policy-Aware Agent Loop with Cedar and OpenClaw" (industry analysis and reference implementation).

These implementations validate that the architectural pattern is technically viable. They are not, individually, full integration profiles in the GAuth credential-bound sense — they integrate a PaC engine with a coding-agent hook, not with a credential-bound delegation chain. CCPE-B specifies the integration pattern that those implementations would extend to in order to gain credential-bound authority enforcement.

3.3 Why a Standalone Spec

GiFo-RFC 0140 §3.8.1 ("Policy-as-Code Engines", under §3.8 Engine Pattern Extensibility) notes that the P*P architecture formalized by RFC 0110 is engine-agnostic and that PaC engines could occupy the AGT-class engine role in CCPE-A's Phase 2 extension point if they satisfied the §4.3 conformance criteria. That note is necessary but not sufficient: the §4.3 AGT-class criteria assume engine-native trust scoring, execution privilege rings, and kill switch capability, which standalone PaC engines do not provide. The §3.8.1 note covers the case where a PaC engine is wrapped to look like an AGT-class engine; it does not cover the case where a standalone PaC engine is integrated as-is, with its missing capabilities supplied externally.

CCPE-B is the spec for the latter case. It specifies (a) what the credential layer (and optionally the Gimel Auth Platform) must supply that the PaC engine does not, (b) how the chaining is shaped when the PaC engine is in its native deployment topology rather than wrapped to look like AGT, and (c) the Power*Point ↔ Policy*Point semantics that make the chain coherent rather than collapsed.

A second motivation for a standalone spec is the boundary surfaced by the Microsoft AGT design itself. AGT supports YAML, Rego, and Cedar as policy languages. Without a CCPE-B spec, the boundary between "AGT-class engine configured with Rego syntax" (still CCPE-A by ownership and operational model) and "standalone OPA deployment with its own bundle server, policy administrators, and pre-existing application embedments" (CCPE-B by ownership and operational model) collapses into a syntax-only distinction. Specifying CCPE-B keeps that boundary architecturally meaningful: CCPE-A and CCPE-B differ not in policy syntax but in the PEP-placement topology, the trust-state ownership model, the multi-tenancy primitive, and the engine-conformance contract.

3.4 Profile Determination — Am I CCPE-A or CCPE-B?

Deployers whose environment combines AGT-class and PaC-class characteristics need a determination procedure to know which profile applies. The boundary is not the policy language; it is the ownership and operational model of the engine, plus whether the engine satisfies the AGT-class conformance criteria of RFC 0140 §4.3 natively (CCPE-A) or via externally supplied PIP feeds (CCPE-B).

Apply the following test in order; the first matching answer determines the profile:

1. **Does the engine satisfy RFC 0140 §4.3 natively** - i.e., does the engine itself maintain trust scoring, execution privilege rings, kill-switch state, and circuit-breaker state, and expose them as engine-native capabilities (not as values supplied to it on each evaluation)?

- “Yes” → “CCPE-A”.
- “No” → continue.

2. **Is the engine deployed as part of an enterprise governance toolkit's runtime** - i.e., is the engine's lifecycle owned by the same operational team that owns the agent runtime, with the engine's policy administration coupled to the toolkit's policy-management surface?

- “Yes” → “CCPE-A”.
- “No” → “CCPE-B”.

The four canonical ambiguous cases and their determination:

```
| Case | Determination | Rationale |

|---|---|---|

| (i) An AGT engine configured with Rego as its policy syntax,
with engine-native trust scoring and rings | **CCPE-A** | Test
```

```
1 = yes. Policy syntax is not a determinant; engine-native
operational governance is. |

| (ii) A standalone OPA daemon wrapped (by the deployer) to
expose AGT-class operational primitives — trust scoring,
rings, kill switch — that are computed inside the wrapper |
**CCPE-A** | Test 1 = yes (post-wrapping). The wrapper makes
the OPA stack AGT-class-conformant by RFC 0140 §4.3 criteria.
|

| (iii) Microsoft AGT with Cedar policies embedded in the AGT
runtime | **CCPE-A** | Test 1 = yes. Cedar is the policy
syntax; AGT remains the engine providing operational
governance. |

| (iv) A managed PaC platform (e.g., Permit.io, Styra DAS)
that supplies a vendor-managed risk score per agent through
its API — but does not expose AGT-class kill-switch / ring /
circuit-breaker primitives natively | **CCPE-B** | Test 1 = no
(vendor-managed risk score is sourced through a PIP-equivalent
surface, not engine-native operational governance). The
deployment uses the trust-state PIP contract of §5.3 to feed
the vendor's score in alongside GAuth-native fields. |
```

Where a deployment cannot make the determination unambiguously (for example, an engine that is engine-native for some AGT-class capabilities and externally supplied for others), the deployment Should pick the profile that matches the “majority” of its operational governance surface and document the partial divergence in its compliance attestation. CCPE-A and CCPE-B are deployment-shape labels, not exclusive product categories.

4. Integration Architecture

4.1 The Power-PEP → Policy-PDP Chain

The CCPE-B integration architecture is a two-stage chain:

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│   Agent action request (verb, resource, context)         │
│                          │                               │
│                          ▼                               │
│   ┌──────────────────────────────────────────┐           │
```

```
|   |  PHASE 1 — Credential-Bound (GAuth)          |       |
|   |                                              |       |
|   |  Power-PEP (RFC 0117, 16-check pipeline)     |       |
|   |    • PoA validity, scope, narrowing          |       |
|   |    • Verb-resource-constraint evaluation     |       |
|   |    • Delegation-chain integrity (CHK-16)     |       |
|   |    • Governance profile ceilings             |       |
|   |                                              |       |
|   |  Result: Phase-1 verdict                     |       |
|   └──────────────────────────────────────────────┘       |
|                          |                               |
|           (if Phase-1 PERMIT and Phase 2 invoked)        |
|                          ▼                               |
|   ┌──────────────────────────────────────────────┐       |
|   |  PHASE 2 — Platform-Bound (PaC engine)       |       |
|   |                                              |       |
|   |  Phase 2 PaC Adapter                         |       |
|   |    • Translates request → engine input       |       |
|   |    • Supplies trust state via PIP            |       |
|   |    • Invokes engine                          |       |
|   |    • Normalizes return → Decision Envelope   |       |
|   |                                              |       |
|   |  Policy-PDP (OPA / Cedar / SpiceDB)          |       |
|   |    • Evaluates platform policy               |       |
|   |    • Returns engine-native decision          |       |
|   └──────────────────────────────────────────────┘       |
|                          |                               |
|                          ▼                               |
|   Combined verdict: PERMIT / DENY / CONSTRAIN            |
|   (CONSTRAIN issued by Power-PEP only;                   |
|    Phase 2 contributes verdict and obligations)          |
|                                                          |
└──────────────────────────────────────────────────────────┘
```

```
```

Phase 1 is unchanged from RFC 0117: GAuth's Power-PEP runs the 16-check pipeline against the PoA credential. Phase 2 is the CCPE-B-specific stage: the Phase 2 PaC Adapter (§4.3) translates the Power-PEP's evaluated request into the form the PaC engine expects, supplies trust state via PIP (§5.3), invokes the engine, and normalizes the response into the Decision Envelope. The combined verdict is issued by the Power-PEP per RFC 0117 §5 (PERMIT / DENY / CONSTRAIN). CONSTRAIN is issued exclusively by the Power-PEP — Phase 2 contributes verdict and structured obligations but does not produce or modify CONSTRAIN.

Note on CONSTRAIN exclusivity (alignment with CCPE-A). The rule that CONSTRAIN is issued exclusively by Phase 1 is identical between CCPE-A (RFC 0140 §2.1, "CONSTRAIN is produced exclusively by PEP Phase 1 - Phase 2 does not produce or modify constraints") and CCPE-B (this RFC). Phase 2 in both profiles contributes a binary verdict plus optional structured obligations; the obligations are consumed by the Power-PEP as inputs to its CONSTRAIN issuance, but are not themselves the CONSTRAIN. This design choice keeps the credential-bound layer as the sole authority for narrowing the agent's scope, irrespective of which Phase 2 engine pattern (AGT-class or PaC) the deployment adopts. The structural property is captured normatively in §7 EC-10.

The architectural property worth naming: The Power-PEP chains outward to a Policy-PDP, never the reverse. PaC engines do not (and should not) call into GAuth as a downstream evaluator within their own evaluation; doing so collapses the Phase 1 / Phase 2 separation and removes the credential-bound authority check from its proper position at the front of the chain. Where a deployment requires the inverse direction (PaC engine asks GAuth a credential question as part of its own evaluation), that is a different integration pattern.

The inverse direction — a Policy-as-Code engine that calls GAuth as a downstream credential PIP from inside its own evaluation — is within the scope of this RFC as a documented but non-conformant integration pattern, designated CCPE-B-NCI (Non-Conformant Inverted Chain). CCPE-B-conformant deployments Must implement the Power-PEP → Policy-PDP chain in the order specified above. Deployments that invert the chain practice the GAuth credential-bound enforcement primitive (and remain subject to the patent and license terms of GiFo-RFC 0101, in preparation, and the MPL-licensed reference implementations) but Must Not be marketed as CCPE-B compliant and are not entitled to use the CCPE family designation. The CCPE-B-NCI designation is normative for §10 correction-channel challenges and for the unified audit trail's phase2_evaluation.profile field, which Must be recorded as ccpe-b-nci rather than ccpe-b for any deployment where the chain is inverted.

4.2 PEP Placement Topologies

The single most consequential difference between CCPE-A and CCPE-B is PEP placement. CCPE-A inherits Microsoft AGT's centralized topology: one `PreToolUse` hook per agent runtime, all tool calls flow through it. CCPE-B integrates with engines whose deployments are typically embedded - every microservice that participates in authorization makes its own SDK call to the PaC engine. Both topologies are valid and both deploy widely; the chaining pattern must accommodate either without forcing rearchitecture of the existing deployment.

Two subprofiles cover the topologies:

4.2.1 CCPE-B / Centralized

The PaC engine is deployed as a single networked service (OPA daemon, SpiceDB cluster) and the agent runtime invokes it through a single Phase 2 PaC Adapter that sits behind the GAuth Power-PEP. Architecturally identical to CCPE-A from the agent runtime's point of view: one hook, one chain. The Phase 2 PaC Adapter is the only CCPE-B-specific component the agent runtime needs to configure.

This subprofile is the easiest CCPE-B onboarding path and is Recommended for greenfield deployments that have no pre-existing PaC embedment.

4.2.2 CCPE-B / Embedded

The PaC engine is invoked as an SDK call from inside multiple downstream applications or microservices. The agent runtime's tool calls fan out to those applications, and each application's existing PaC embedment is preserved. The integration pattern here is:

- **The Power-PEP runs once at the agent runtime**, applying Phase 1 credential checks against the PoA credential carried by the agent.
- **The Power-PEP issues a normalized Phase 1 attestation** (a signed assertion that the agent's mandate permits this action class, with the relevant constraints) carried alongside the action request to each downstream application.
- **Each downstream application's existing PaC embedment** receives both the action and the Phase 1 attestation, supplies the attestation as PIP input to its local PaC engine evaluation, and produces its own Phase 2 decision.
- **The Power-PEP does not invoke the embedded PaC engines directly**. The chain is realized cooperatively across the agent runtime and the downstream applications.

This subprofile preserves existing PaC deployments without forcing reconfiguration. It is Recommended for brownfield deployments where the PaC engine is already embedded across many services.

4.2.2.1 Phase 1 Attestation Format

The Phase 1 attestation is a JWT (RFC 7519) signed by the Power-PEP and delivered to each downstream application alongside the action request. The attestation's required and optional claims are:

| Claim | Type | Required | Description |

|---|---|---|---|

| `iss` | string (URI) | Required | The Power-PEP issuer identifier (the GAuth deployment's authoritative URL). |

| `sub` | string (URI) | Required | The agent identifier (matches `mandate.subject` in the underlying PoA credential). |

| `aud` | string or array of strings | Required | The downstream application identifier(s) authorized to consume this attestation. Single-audience attestations are Recommended. |

| `iat` | integer (Unix epoch seconds) | Required | Issued-at time. |

| `exp` | integer (Unix epoch seconds) | Required | Expiry. Recommended ≤ 60 seconds from `iat` to bound replay window (see §8). |

| `jti` | string | Required | Unique identifier for this attestation, used for replay-detection by downstream applications. |

| `cnf` | object | Required | Confirmation claim per RFC 7800 binding the attestation to the underlying PoA credential. Carries `{"poa_ref": "<PoA credential URI>", "poa_thumbprint": "<base64url SHA-256 of canonicalized PoA credential>"}`. |

| `gauth_action` | object | Required | The verb-resource-constraint Phase 1 evaluated. Shape: `{"verb": "<verb URI>", "resource": "<resource URI>", "constraint": {...}}`. |

| `gauth_phase1_verdict` | enum {`permit`, `deny`} | Required | The Phase 1 verdict. Attestations carrying `deny` Must Not be issued by the Power-PEP — the production of a signed Phase

```
1 attestation is itself a Phase-1-PERMIT signal — but the
field is normative for parser strictness. |

| `gauth_trust_state` | object | Required | Snapshot of the
trust-state PIP fields (§5.3) at the moment Phase 1 evaluated.
Downstream PaC engines consume this snapshot rather than re-
querying GAuth. |

| `gauth_phase2_required` | boolean | Optional | Per-mandate
fail-closed posture for the Phase-1-healthy / Phase-2-degraded
case (§8). **Defaults to `true`** (fail-closed, aligned with
RFC 0140 v1.2 §11.5). When omitted, downstream applications
Must treat the value as `true`. When explicitly set to
`false`, a Phase-2 degradation Must produce a combined-verdict
PERMIT consistent with the Phase-1 verdict — this is an opt-in
availability posture for deployments where the credential-
bound Phase-1 layer is the authoritative regulatory
enforcement and Phase 2 is purely defense-in-depth (see §8
prose). |
```

The attestation Must be cryptographically bound to the PoA credential via the `cnf` claim, to the action class via `gauth_action`, to a short TTL via `exp`, and to the audience via `aud`. The combination of these bindings is what prevents an attestation issued for action A against application X from being replayed against action B or against application Y. Detailed binding-mechanism requirements (signing key management, key rotation, attestation-signing-key publication) are enumerated in §8 (Security Considerations).

Downstream applications Must validate `iss`, `aud`, `exp`, and the `cnf` binding before consuming the attestation as PaC input. Applications Should additionally maintain a short-lived `jti`-replay cache sized to the attestation TTL to detect replays within the validity window.

The attestation is a normative artifact of the CCPE-B / Embedded subprofile. Implementations are not required to use JWT specifically; equivalent signed-envelope formats (CWT per RFC 8392, SD-JWT) are permitted provided the same set of bound fields is preserved with equivalent cryptographic guarantees. JWT is Recommended for ecosystem alignment with RFC 0116 Extended Tokens.

4.2.3 Hybrid

Mixed deployments — one centralized PaC engine for some action classes, embedded PaC engines for others — are permitted. The Power-PEP routes each action to its appropriate Phase 2 evaluator according to deployment configuration. No CCPE-B-

specific machinery is required beyond the routing rule; the routing rule is itself a deployment decision, not a normative requirement of this RFC.

4.3 Decision-Shape Adapters

PaC engines return heterogeneous decision shapes. The Phase 2 PaC Adapter normalizes them into the Decision Envelope `{verdict, obligations, diagnostics}` that the Power-PEP consumes. The minimum normalization contract per engine:

```
| Engine | Native return | Normalization to Decision Envelope
|

|---|---|---|

| **OPA / Rego** | Arbitrary structured Rego result (often a
document with `allow`, `deny`, and policy-specific fields) |
The adapter Must specify which Rego document path constitutes
`verdict` (typically a boolean `allow` or `deny` field), which
path constitutes `obligations` (typically a structured object
of constraint fields), and which path constitutes
`diagnostics`. The mapping is policy-author-controlled and
Must be documented per policy bundle. |

| **Cedar** | `Allow` or `Deny` decision with optional
`diagnostics` (errors, reasons, satisfying policies) |
`verdict` maps directly. `obligations` are typically empty for
Cedar (Cedar is binary by design); structured constraints, if
needed, are carried in policy-specific entity attributes the
policy author exposes via context. `diagnostics` map directly.
|

| **SpiceDB** | Boolean `check` result; optional `expand`
permission tree | `verdict` maps from the boolean `check`
result. `obligations` are typically empty. `diagnostics` may
carry the `expand` tree where the deployment chooses to
surface it for audit. |
```

Two normalization invariants Must hold:

1. **Semantic preservation**. A native engine `Deny` Must normalize to `verdict = deny`. A native engine `Allow` Must normalize to `verdict = permit`. Engine-specific intermediate states (e.g., Rego policies that return neither `allow = true` nor `deny = true`) Must be normalized to a documented default (Recommended default: `verdict = deny`) — never silently elided.

2. **Obligation faithfulness**. Where an engine surfaces structured obligations (Rego-extracted constraint objects, Cedar entity attributes the policy intentionally exposes), the adapter Must pass them through unmodified to the Power-PEP, which will apply them as the basis for issuing CONSTRAIN at the combined-verdict step.

The adapter Should NOT inject novel obligations the engine did not produce. The adapter is a translation layer, not a policy author.

Limitation - Cedar obligations channel. Cedar is a binary-decision engine by design. CCPE-B / Cedar deployments cannot carry structured Phase-2 obligations through the Decision Envelope; the `obligations` slot will be empty in normal operation. This means CCPE-B / Cedar deployments cannot drive Power-PEP CONSTRAIN through the Phase-2 obligations channel and Must rely on Phase-1 narrowing (PoA scope, governance profile ceilings, mandate constraints) for any post-Phase-2 narrowing of the agent's authority. Deployments that need fine-grained Phase-2-driven CONSTRAIN should select an engine whose decision shape supports structured obligations (OPA / Rego), or carry the constraint metadata via Cedar policy-context entity attributes the deployment commits to surfacing through a policy convention. See also §7 EC-03 (Granularity) and §C.4 (Cedar adapter wiring notes).

4.3.1 Mapping the Decision Envelope to the RFC 0140 §9 Unified Audit Record (Normative)

RFC 0140 §9 specifies the unified audit record schema for CCPE-A (G-AGT) deployments and is referenced by this RFC as the single compliance-ready audit stream for any CCPE deployment. The `phase2_evaluation` block of that schema was originally specified against an AGT-class binary engine. CCPE-B Phase-2 evaluators produce a richer Decision Envelope (`{verdict, obligations, diagnostics}`) per §4.3, and an organization that runs both CCPE-A and CCPE-B deployments must be able to reassemble a single audit picture across the two profiles. This subsection specifies the normative mapping from the Decision Envelope into the RFC 0140 §9 `phase2_evaluation` block so that audit consumers can correlate the two streams without per-deployment custom translation. The mapping is profile-additive: it only populates fields inside `phase2_evaluation` (and, for the `phase2_required` failure-mode flag, `combined_decision`); no other section of the RFC 0140 §9 record is altered.

4.3.1.1 Canonical Field Mapping

The Phase 2 PaC Adapter Must populate the following fields in the unified audit record per evaluation. Field paths are relative to the RFC 0140 §9.2 schema. New fields introduced by this subsection are marked `(new in CCPE-B)`; reused fields adopt the RFC 0140 §9.3 semantics.

| Decision Envelope source | Audit record field | Type | Population rule |

|---|---|---|---|

| `verdict` | `phase2_evaluation.decision` | enum `{ALLOW, DENY, SKIPPED, ERROR}` | `permit` → `ALLOW`; `deny` → `DENY`. `SKIPPED` Must be used when Phase 1 returned DENY (per RFC 0140 §9.3 short-circuit rule). `ERROR` Must be used when the engine could not be evaluated (Phase-2 degraded row of §8); see §4.3.1.3. |

| `obligations` | `phase2_evaluation.obligations` *(new in CCPE-B)* | array of structured constraint objects | The adapter Must pass through the engine-surfaced obligations object unmodified per the §4.3 obligation-faithfulness invariant. `[]` (empty array) when the engine surfaced no obligations or the engine cannot surface obligations (Cedar, SpiceDB; see §4.3.1.2). The adapter Must Not synthesize obligations the engine did not produce. |

| `diagnostics` | `phase2_evaluation.diagnostics` *(new in CCPE-B)* | object | Engine-native diagnostics surface (see §4.3.1.2 for per-engine shape). May be `{}` when the engine produced no diagnostics. Implementations Should not omit the field entirely (set to `{}`) so consumers can rely on its presence. |

| n/a — adapter context | phase2_evaluation.profile (new in CCPE-B) | enum {ccpe-a, ccpe-b, ccpe-b-nci, ccpe-b-ncp, ccpe-b-nco} | Must be ccpe-b for any record produced by a Phase 2 PaC Adapter operating in conformance with §4.1. Non-conformant variants Must populate the corresponding designation per §5.4.1. CCPE-A records continue to omit this field or set it to ccpe-a. This is the primary key audit consumers use to correlate cross-profile streams and to detect non-conformant deployments. |

| n/a — adapter context | `phase2_evaluation.engine` *(new in CCPE-B)* | enum `{opa, cedar, spicedb, other}` | The engine family that produced the Decision Envelope. Required when `profile = ccpe-b`. `other` is permitted for engines admitted via §10 (Engine Onboarding); the deployment Must additionally populate `engine_family_ref` with a stable identifier. |

| n/a — adapter context | `phase2_evaluation.engine_version` | string | Already present in RFC 0140 §9.2; under CCPE-B this

is the PaC engine version (OPA / Cedar / SpiceDB), not the AGT engine version. |

| n/a — adapter context | `phase2_evaluation.policy_artifact_ref` *(new in CCPE-B)* | string | Stable reference to the bundle / policy store / schema version evaluated (e.g., OPA bundle digest, Cedar policy-set version, SpiceDB schema revision). Required so that §8's "bundle / store / namespace version evaluated for each decision matches the version expected for the deployment's release manifest" check is auditable post-hoc. |

| n/a — §8 routing | `phase2_evaluation.evaluator_id` *(new in CCPE-B)* | string | In Hybrid topology (§4.2.3) the identifier of the specific Phase 2 evaluator that handled the decision. Required when the deployment runs more than one Phase 2 evaluator; satisfies the §8 multi-engine consistency requirement that the unified audit trail "Should record which Phase 2 evaluator handled each decision so that routing-rule errors are detectable post-hoc". |

| `phase2_required` (per-mandate, from §8 / §4.2.2.1) | `phase2_evaluation.phase2_required` *(new in CCPE-B)* | boolean | The value of the per-mandate `phase2_required` flag carried on the PoA constraint object (or `false` if absent). Required so audit consumers can interpret the failure-mode rows of §8. |

| derived (see §4.3.1.3) | `phase2_evaluation.failure_mode` *(new in CCPE-B)* | enum `{none, phase2_degraded_fallback, phase2_degraded_denied, phase1_degraded}` | Must be `none` when both phases evaluated normally. Must be set per the failure-mode table in §4.3.1.3. The field Must always be present and Must Not be `null`; `none` is the canonical normal-state encoding so audit consumers can rely on a single value to test against. |

| derived | `combined_decision.phase2_required_honored` *(new in CCPE-B)* | boolean | `true` when the combined verdict was determined under `phase2_required = true` semantics (i.e., the fail-closed default applied at fall-back time, including the case where the flag was absent and treated as `true` per §4.2.2.1). `false` when the per-mandate opt-in (`phase2_required = false`) caused the Phase-1 verdict to be accepted. Distinguishes "fail-closed-default DENY" from "opt-in fall-back PERMIT"; see §4.3.1.3. |

The existing RFC 0140 §9.2 fields `phase2_evaluation.rules_evaluated`, `rules_matched`, `processing_time_ms`, `violated_rules`, `execution_ring`, and `ring_override` remain optional under CCPE-B. They were defined for AGT-class engines and do not have universal counterparts across OPA / Cedar / SpiceDB; per-engine population rules are given in §4.3.1.2. The `scoring` block of the RFC 0140 §9.2 record is unchanged: when a CCPE-B deployment feeds an `agent.trust_score` to the engine via the §5.3 trust-state PIP, that score Must be recorded in `scoring.trust_score` exactly as it was passed to the engine, and `scoring.trust_score_method` Must remain `"deterministic"`. CCPE-B does not introduce a confidence-score path.

4.3.1.2 Per-Engine Population Profile

Different PaC engines populate different fields. The table below specifies, for each of the three exemplar engines, which Decision Envelope slots are populated (`P`), structurally empty (`E`), or carry engine-native diagnostics (`D`), and pins the diagnostics shape so audit consumers can parse the `diagnostics` object without per-deployment knowledge of the engine.

Field	OPA / Rego	Cedar	SpiceDB
`phase2_evaluation.decision` (from `verdict`)	**P** — from the policy-author-designated `allow` / `deny` document path per §4.3.	**P** — direct mapping from Cedar `Allow` / `Deny`.	**P** — from the boolean `check` result.
`phase2_evaluation.obligations` (from `obligations`)	**P** — from the policy-author-designated obligations document path per §4.3; pass-through, unmodified.	**E** — Must be `[]`. The §4.3 limitation callout applies: Cedar cannot carry structured obligations through the envelope. The adapter Must Not synthesize obligations from policy-context attributes; deployments that surface constraint metadata via context attributes Must do so under a published policy convention and record those attributes in `diagnostics.cedar.context_attributes_surfaced` (see below) rather than in `obligations`.	**E** — Must be `[]`. SpiceDB returns a boolean check; structured obligations are out of scope.
`phase2_evaluation.diagnostics` (from `diagnostics`)	**D** — object with at minimum `{rego_query, document_paths_evaluated, decision_log_id}`. The `decision_log_id` Must reference the OPA decision log entry for the evaluation when the deployment has decision logging		

enabled. `rules_evaluated` and `rules_matched` Should be populated in the parent `phase2_evaluation` block when the policy bundle exposes them. `violated_rules` May carry the policy-author-designated denial reason paths. | **D** — object with at minimum `{determining_policies, errors, reasons}`, mapping directly from Cedar's diagnostics surface. `context_attributes_surfaced` (optional) carries the policy-convention-published context attributes the deployment uses in lieu of structured obligations. `rules_evaluated` / `rules_matched` Should map to Cedar's policy-set evaluation count when available. | **D** — object with at minimum `{check_consistency, expand_tree}`. `expand_tree` is populated only when the deployment has opted into surfacing expansion for audit; otherwise omit. `check_consistency` Must record the SpiceDB consistency level used for the check (`minimize_latency`, `at_least_as_fresh`, `at_exact_snapshot`, `fully_consistent`). The RFC 0140 §9.2 `rules_evaluated` / `rules_matched` fields are not meaningful for SpiceDB and Should be omitted. |

| `phase2_evaluation.engine` | `opa` | `cedar` | `spicedb` |

| `phase2_evaluation.policy_artifact_ref` | OPA bundle digest (e.g., `sha256:…`) and bundle name. | Cedar policy-set version identifier. | SpiceDB schema revision (Zedtoken or migration revision). |

The `obligations` slot for Cedar and SpiceDB is structurally empty by engine design, not by accident. Audit consumers Must therefore not infer anything from `obligations = []` on a Cedar or SpiceDB record beyond "the engine cannot carry structured obligations". On an OPA record, `obligations = []` indicates the policy did not surface any.

4.3.1.3 Failure-Mode Mapping and `phase2_required` Distinguishability

The §8 failure-mode table defines five rows. This subsection expands the Phase-1-healthy / Phase-2-degraded row into the two `phase2_required` cases (six rows total below) so the audit record can distinguish a fail-closed enforcement (default) from a deployment-opted-in PERMIT fall-back, allowing an audit consumer to answer the question "was this combined verdict produced under the fail-closed default or under the opt-in availability posture?" without re-deriving the deployment's risk posture from out-of-band configuration. The mapping is:

| §8 failure row | `pep_evaluation.decision` | `phase2_evaluation.decision` | `phase2_evaluation.failure_mode` |

```
`phase2_evaluation.phase2_required` |
`combined_decision.decision` |
`combined_decision.phase2_required_honored` |

|---|---|---|---|---|---|---|

| Both healthy, Phase 2 permit | `PERMIT` | `ALLOW` | `none` |
`false` or `true` | `PERMIT` | `false` |

| Both healthy, Phase 2 deny | `PERMIT` | `DENY` | `none` |
`false` or `true` | `DENY` | `false` |

| Phase 1 deny | `DENY` | `SKIPPED` | `none` | `false` or
`true` | `DENY` | `false` |

| Phase 1 healthy, Phase 2 degraded, `phase2_required = false`
| `PERMIT` | `ERROR` | `phase2_degraded_fallback` | `false` |
`PERMIT` | `false` |

| Phase 1 healthy, Phase 2 degraded, `phase2_required = true`
| `PERMIT` | `ERROR` | `phase2_degraded_denied` | `true` |
`DENY` | `true` |

| Phase 1 degraded | `ERROR` | (any, including `SKIPPED`) |
`phase1_degraded` | `false` or `true` | `DENY` | `false` |
```

Two rows differ only in `phase2_required` and the resulting combined verdict. The `combined_decision.phase2_required_honored` boolean is the audit-consumer-visible signal that distinguishes the two cases. A combined-verdict `DENY` paired with `phase2_required_honored = true` is a fail-closed enforcement under the default posture (RFC 0140 v1.2 §11.5-aligned); a combined-verdict `PERMIT` paired with `phase2_evaluation.failure_mode = phase2_degraded_fallback` is an opt-in fall-back PERMIT that the deployment has explicitly enabled by setting `phase2_required = false` on the mandate. Audit consumers Must Not coalesce these two cases.

Where the engine produced no Decision Envelope at all (engine unreachable, bundle corruption, schema migration in flight), `phase2_evaluation.decision` Must be `ERROR`, `obligations` Must be `[]`, and `diagnostics` Must carry an `error` object with at minimum `{code, message}` where `code` follows the RFC 0140 §5.1.4 error-code conventions (`AGT_UNREACHABLE` Should be replaced under CCPE-B with the more accurate `PHASE2_ENGINE_UNREACHABLE`, `PHASE2_BUNDLE_CORRUPT`, `PHASE2_SCHEMA_MIGRATION`, or `PHASE2_EVALUATION_ERROR`).

4.3.1.4 Cross-Profile Reassembly

A deployment running both CCPE-A (G-AGT) and CCPE-B (PaC) reassembles a single audit picture per the following rules:

1. The `phase2_evaluation.profile` field is the partitioning key: `ccpe-a` records carry the AGT-class fields (`rules_evaluated`, `execution_ring`, `ring_override`); `ccpe-b` records carry the Decision Envelope fields (`obligations`, `diagnostics`, `engine`, `policy_artifact_ref`, `evaluator_id`, `phase2_required`, `failure_mode`).
2. All other RFC 0140 §9 fields — `record_id`, `timestamp`, `action`, `agent`, `delegation`, `pep_evaluation`, `scoring`, `combined_decision.decision`, `metadata.correlation_id` — carry identical semantics across both profiles and are the join keys for cross-profile correlation.
3. `combined_decision.constrain_provenance` is unchanged: CONSTRAIN is exclusive to Phase 1 in both profiles (§4.1, EC-10).
4. The §8 multi-engine consistency requirement is satisfied when, for any Hybrid-topology deployment, every record carries a populated `phase2_evaluation.evaluator_id` and the deployment's release manifest enumerates the expected `evaluator_id` ↔ action-class routing rules.

This subsection is normative for any Phase 2 PaC Adapter that emits records into an RFC 0140 §9 unified audit trail. Deployments that emit Phase 2 audit data into a separate, non-RFC-0140 audit stream are not bound by §4.3.1, but are not CCPE-B conformant for cross-profile reassembly purposes (§10).

4.4 Adapter Type Reuse from RFC 0140

GAuth's connector model defines a 7-slot adapter framework with four adapter types (RFC 0140 §2.2):

- **Type A** - OAuth-engine-style vehicle adapters (Slot 2).
- **Type B** - Action-execution and secrets-management adapters (Slots 3, 4).
- **Type C** - Proprietary capability adapters (Slots 5, 6, 7) — Gimel Technologies sealed.
- **Type D** - Future-reserved.

The Phase 2 PaC Adapter (§4.3) does not occupy a Type A/B/C connector slot. Like the AGT-class engine adapter in RFC 0140, it integrates via the **PEP Phase 2 extension point** (RFC 0140 §4.2), which is architecturally distinct from the connector slot model and is not subject to tariff gating, sealed-manifest attestation, or connector slot registration. The Phase 2 PaC Adapter's runtime lifecycle is owned by the deployment that operates the PaC engine — typically the platform-engineering or IAM team that maintains the existing PaC embedment.

Where a deployment additionally connects to Gimel Auth Platform proprietary services (managed trust-state PIP feeds, AI-enhanced threat detection corpus, managed

governance adapter), those connections occupy the Type C slots per RFC 0140 §8 and are subject to the proprietary licensing terms of those slots. The Phase 2 PaC Adapter itself remains open-core.

4.4.1 Open-Core Boundary and the Type-C Dependency

The "open-core" framing of the Phase 2 PaC Adapter applies to the **adapter shell** (request translation, engine invocation, decision-envelope normalization). It does not automatically extend to every field a downstream policy may reference, because the trust-state PIP contract (§5.3) sources several optional fields from Gimel Auth Platform Type-C connectors. Pure-OSS deployers should be explicit with themselves about which fields they will populate from open-source computation versus from a Type-C connection:

```
| Trust-state field (§5.3) | Required? | Open-core source
available? | Type-C source |

|---|---|---|---|

| `mandate.ref` | Required | Yes — produced by GAuth's Power-
PEP from the PoA credential. | n/a |

| `mandate.governance_profile` | Required | Yes — RFC 0115 §4.
| n/a |

| `mandate.delegation_depth` | Required | Yes — RFC 0117 / RFC
0115. | n/a |

| `mandate.budget_remaining` | Required | Yes — RFC 0118
budget semantics. | n/a |

| `agent.kill_switch_state` | Required | Yes — produced by
GAuth's open-core operational-controls layer. | n/a |

| `agent.circuit_breaker_state` | Required | Yes — produced by
GAuth's open-core operational-controls layer. | n/a |

| `agent.trust_score` | Optional | Yes, *if* the deployer
implements deterministic trust-score computation per RFC 0140
§4.3.3 themselves. The reference implementation of
deterministic 5-tier trust scoring is bundled with the Gimel
Auth Platform Type-C distribution; pure-OSS deployers may roll
their own deterministic equivalent. | Slot 5 (managed
deterministic scoring as part of the GovernanceAdapter
bundle). |
```

```
| `agent.privilege_ring` | Optional | Yes, *if* the deployer
maintains ring assignment themselves. | Slot 5 (managed ring
assignment). |

| `confidence_score` | Optional | **No** — confidence scoring
is an AI-enhanced capability and is exclusively a Type-C /
Slot 5 capability per RFC 0140 §1.1 Exclusions. Pure-OSS
deployers leave this field null. | Slot 5 (proprietary
GovernanceAdapter). |
```

A pure-OSS CCPE-B deployment is fully viable using only the Required fields plus open-core implementations of the Optional fields. Connecting Type-C is a deployment choice, not a CCPE-B prerequisite. The Phase 2 PaC Adapter shell, the trust-state PIP contract definition, the Phase 1 attestation format, and the engine cookbooks (Appendices B–D) are all open-core and remain so independent of any Type-C decision.

5. Trust-State Externalization and CCPE-B Engine Conformance

5.1 What PaC Engines Provide Natively

The exemplar PaC engines provide, as published by their maintainers:

- **Deterministic policy evaluation** against a documented policy language (Rego, Cedar) or relationship model (SpiceDB).
- **Structured decision return** (per §4.3).
- **Multi-tenant policy isolation** via engine-native primitives (per §6).
- **Audit logging** of evaluation requests and decisions (engine-dependent in shape and detail).

These capabilities are sufficient for CCPE-B Phase 2 evaluation as long as the PIP contract in §5.3 is met.

5.2 What PaC Engines Do Not Provide Natively

The exemplar PaC engines do not, as a rule, provide:

- **Native trust scoring**. PaC engines evaluate policy against input; they do not maintain a behavioral score per agent over time. (Where a deployment wishes to use a trust score in a PaC policy, the score must be supplied as policy input.)
- **Native execution privilege rings**. PaC engines return a decision; they do not gate execution by ring assignment. (Where a deployment wishes to model rings, ring assignment must be supplied as policy input and the policy must reference it.)
- **Native kill switch**. PaC engines have no notion of an out-of-band kill state for an agent that overrides ordinary policy evaluation. (Where a deployment wishes to

honor a kill switch, the kill state must be supplied as policy input and the policy must include a top-of-evaluation deny rule keyed to it.)

- **Native circuit breaker**. Same property as kill switch: must be supplied as input and referenced by policy.
- **Native cascade revocation**. PaC engines do not propagate revocations across delegation chains. (CCPE-B handles this in Phase 1 via GAuth's mandate revocation; the PaC engine sees the revocation only as the absence of a valid Phase 1 attestation in the Embedded subprofile, or as a top-level Phase-1 deny in the Centralized subprofile.)

These absences are not deficiencies of PaC engines. They are scope boundaries: PaC engines were designed to evaluate policy, not to maintain operational governance state. Where a deployment requires the operational-governance capabilities listed above, CCPE-B specifies that they are supplied by the credential layer (via the trust-state PIP contract in §5.3) or by the Gimel Auth Platform as a proprietary value-add. The PaC engine consumes them as inputs; it is not asked to produce them.

5.3 Trust-State PIP Contract

The trust-state PIP contract specifies the input the Phase 2 PaC Adapter Must supply to the PaC engine on each evaluation, sourced from GAuth and/or the Gimel Auth Platform. The minimum trust-state object is:

```
| Field | Source | Type | Required? | Notes |

|---|---|---|---|---|

| `mandate.ref` | GAuth | string (URI) | Required | Reference
to the PoA credential under which the action is being
attempted. |

| `mandate.governance_profile` | GAuth | string | Required |
One of the five governance profiles per RFC 0115 §4. |

| `mandate.delegation_depth` | GAuth | integer | Required |
Current depth in the delegation chain. |

| `mandate.budget_remaining` | GAuth | object | Required | Per
RFC 0118 budget semantics. |

| `agent.trust_score` | Gimel Auth Platform | integer (0–1000)
or null | Optional | Where a deterministic trust score is
maintained. See §5.3.1 for the normative ring/score model.
Null if not maintained. |

| `agent.privilege_ring` | Gimel Auth Platform | integer (0–3)
or null | Optional | Where execution privilege rings are
```

```
maintained. See §5.3.1 for the normative ring model and the
numbering convention. Null if not maintained. |

| `agent.kill_switch_state` | GAuth | enum {`nominal`,
`tripped`} | Required | Top-level kill state. `nominal` means
the kill switch is **not** engaged and the agent is operating
normally. `tripped` means the kill switch is engaged; the
policy Should evaluate to `deny` regardless of other inputs.
The `nominal` / `tripped` naming is deliberate: it removes the
ambiguity of an "active" value that operators have
historically read as "halted". |

| `agent.circuit_breaker_state` | GAuth | enum {`closed`,
`open`, `half_open`} | Required | Per-agent circuit-breaker
state. Policy consumption is deployment-defined. |

| `confidence_score` | Gimel Auth Platform (Type C) | float
(0.0–1.0) or null | Optional | Where the proprietary
GovernanceAdapter (RFC 0140 §8 Slot 5) is connected. Null
otherwise. Must Not be confused with `agent.trust_score` — see
RFC 0140 §4.3.3 / §7.4. |
```

Additional trust-state fields May be included by deployment agreement. The contract is extensible.

The Phase 2 PaC Adapter is responsible for supplying this object as input to the PaC engine on each evaluation. The PaC engine policy is responsible for referencing whichever fields it needs; fields it does not reference are evaluated as if absent (no implicit policy effect from unreferenced inputs).

5.3.1 Ring Model and Score-to-Tier Mapping (Normative for CCPE-B)

The cookbooks in Appendices B–D and any conformant CCPE-B deployment Must use the following ring-and-profile mapping unless the deployment publishes a divergence in its compliance attestation. This table is reproduced verbatim from RFC 0140 v1.2 §7.1 (Governance Profile → Trust Boundary Mapping) and is the single normative source within RFC 0150; the cookbooks reference this table rather than embedding their own thresholds.

Ring numbering convention. CCPE-B uses a four-ring scale, numbered 0..3, with Ring 0 being the highest privilege (most autonomous) and Ring 3 being the lowest privilege (most constrained). This convention matches RFC 0140 v1.2 §7.1 and is preserved here for cross-profile (CCPE-A / CCPE-B) audit-record consistency. Deployments that integrate with internal Gimel Auth components using a different ring numbering (for example, internal artifacts that count from lowest privilege upward) Must perform an

explicit translation at the trust-state PIP boundary; CCPE-B does not attempt to harmonize with internal-only naming.

Governance Profile → Default Ring (per RFC 0140 v1.2 §7.1). Where a deployment provisions an agent without an explicit ring, the default ring and trust-score floor are derived from the mandate's governance profile (RFC 0115 §4) per the following table. Deployments May override the ring on a per-agent basis through the trust-state PIP feed (subject to the deployment's own approval workflow).

| Governance Profile (RFC 0115 §4) | Default Ring | Trust Score Floor | Oversight Mode | Notes |

|---|---|---|---|---|

| `minimal` | **Ring 0** (highest privilege) | `trust_score ≥ 500` | autonomous | Most autonomous; no human-in-the-loop required. Resource sensitivity ≤ 4. |

| `standard` | **Ring 1** | `trust_score ≥ 400` | supervised | Session-bounded supervision. Resource sensitivity ≤ 3. |

| `strikt` | **Ring 1** | `trust_score ≥ 500` | supervised | Tighter session bounds than `standard`; same ring. Resource sensitivity ≤ 3. |

| `enterprise` | **Ring 2** | `trust_score ≥ 600` | supervised | Default for enterprise mandates. Resource sensitivity ≤ 2. |

| `behoerde` | **Ring 2** | `trust_score ≥ 700` | four-eyes | Highest oversight intensity; four-eyes approval required. Resource sensitivity ≤ 2. |

| *(below profile floor)* | n/a | below applicable floor | n/a | Combined verdict Should be DENY irrespective of policy; the agent's trust score has fallen below the floor required by its own profile. |

Ring 3. RFC 0140 v1.2 §7.1 does not assign Ring 3 as a default for any governance profile. Ring 3 remains a valid explicit-deployment-downgrade target for agents that the operator wishes to constrain below the floor of their nominal profile (for example, during probation or post-incident remediation). When a deployment places an agent at Ring 3, the trust-score floor and oversight mode are deployment-defined; CCPE-B specifies only the resource-sensitivity ceiling rule below.

Resource-sensitivity ceiling rule (CCPE-B-specific addition on top of RFC 0140 v1.2 §7.1). RFC 0140 does not define resource sensitivity. CCPE-B adds the following rule for engine policies that gate access on a resource-sensitivity attribute. Resource sensitivity is numbered on a 1..4 scale where higher S means more sensitive. An agent in Ring R May invoke a resource of sensitivity S only if `S ≤ 4 - R` (equivalently `R + S ≤ 4`). This

produces the privilege-correct mapping: a Ring 0 (highest-privilege) agent May access resources of sensitivity 1..4 (any), a Ring 1 agent May access sensitivity 1..3, a Ring 2 agent May access sensitivity 1..2, and a Ring 3 (lowest-privilege) agent May access only sensitivity 1. The cookbooks in Appendices B–D implement this rule consistently.

Alignment with RFC 0140 v1.2 §7.1. The ring-and-profile mapping above is intentionally identical to the parent spec so that CCPE-A and CCPE-B audit records (RFC 0140 §9 unified audit trail) remain comparable across the two profiles. Where a deployment has historically used the inverted convention (Ring 0 = `behoerde`, Ring 3 = `minimal`) — for example, derived from prior contributor implementations surveyed in §3.2 — that deployment Must perform a one-time translation at the trust-state PIP boundary on adoption of v0.9.5 and Should record the translation in its compliance attestation. The score floors and ring placements above bind on the *governance profile*, not on the ring number alone; the prior per-ring score-floor table (v0.9.4 and earlier) is withdrawn.

5.4 CCPE-B Engine Conformance Criteria

5.4.1 Non-Conformant Architectural Variants — In-Scope Designations

A deployment that practices the GAuth credential-bound enforcement primitive but rearranges the chain in a manner inconsistent with §4.1 is within the scope of this RFC as a documented non-conformant variant. Such deployments are subject to the patent and license terms of the GAuth credential-bound enforcement primitive (see GiFo-RFC 0101, in preparation, and the MPL-2.0 reference implementations) and remain governed by §10 (How to Suggest a Correction); they Must Not be marketed as CCPE-B compliant and are not entitled to use the unqualified CCPE family designation. The following non-conformant designations are normative for §10 challenges and for the phase2_evaluation.profile field of the unified audit trail (§4.3.1.1):

```
| Designation | Pattern | Conformance posture | Patent /
license posture |

|---|---|---|---|

| **CCPE-B** | Power-PEP → Policy-PDP chain as specified in
§4.1 | Conformant | Practices the credential primitive;
license/patent terms apply |

| **CCPE-B-NCI** (Non-Conformant Inverted Chain) | Policy-PDP
→ Power-PEP-as-PIP — the PaC engine calls GAuth as a
downstream credential PIP from inside its own evaluation,
collapsing the Phase 1 / Phase 2 separation and removing the
credential-bound authority check from its proper position at
the front of the chain | Non-conformant | Practices the
credential primitive; license/patent terms apply |
```

| **CCPE-B-NCP** (Non-Conformant Parallel Evaluation) | Phase 1 and Phase 2 evaluated in parallel with no chaining; the combined verdict computed by an out-of-band reconciler that is neither the Power-PEP nor the Policy-PDP | Non-conformant | Practices the credential primitive; license/patent terms apply |

| **CCPE-B-NCO** (Non-Conformant Phase-2 Override) | Phase 2 permitted to override a Phase 1 verdict (in particular, to upgrade Phase 1 DENY to combined PERMIT, or to issue or modify CONSTRAIN in violation of §4.1 and §7 EC-10) | Non-conformant | Practices the credential primitive; license/patent terms apply |

| *(outside the CCPE family)* | Pure Policy-as-Code deployment with no credential-bound layer at any point in the evaluation flow | Outside the family — not a CCPE deployment, no claim on the family designation | Does not practice the credential primitive |

Audit recording. A deployment whose architecture matches one of the non-conformant designations Must populate the `phase2_evaluation.profile` field (§4.3.1.1) with the corresponding designation (`ccpe-b-nci`, `ccpe-b-ncp`, `ccpe-b-nco` — enum values are lowercase) rather than `ccpe-b`. Audit consumers Must treat any record carrying a non-conformant designation as evidence that the deployment is operating outside the CCPE-B conformance contract; downstream certification, attestation, and procurement processes Should treat such records accordingly.

Relationship to §4.1 chain semantics. The architectural property in §4.1 — that the Power-PEP chains outward to a Policy-PDP and the inverse direction collapses the Phase 1 / Phase 2 separation — is restated here as the normative basis for the CCPE-B-NCI designation. Non-conformant deployments are explicitly governed by this RFC rather than disclaimed.

5.4.2 CCPE-B conformance

A PaC engine is CCPE-B conformantif it satisfies all of the following:

a. **Deterministic policy evaluation**. Given the same input, the engine returns the same decision. Engines that incorporate non-deterministic evaluation steps (e.g., AI-enhanced policy generation at evaluation time) are out of CCPE-B scope until they expose a deterministic-evaluation mode.

b. **Multi-tenant policy isolation**. The engine supports per-tenant policy bundling, policy-store isolation, or relationship-namespace isolation such that cross-tenant policy contamination is structurally prevented. (See §6 for engine-specific patterns.)

c. **Structured decision return**. The engine returns a decision in a form that can be normalized into the Decision Envelope per §4.3. Engines that return only opaque allow/deny without diagnostics May be CCPE-B conformant in a degraded mode (no audit detail), but are Recommended to expose at least a minimum diagnostics surface.

d. **PIP-fed input acceptance**. The engine accepts arbitrary structured input on each evaluation call and exposes it to policy evaluation. (All exemplar engines satisfy this trivially; the criterion exists to exclude engines that operate on policy alone with no per-call input.)

e. **Audit capability**. The engine exposes a logging or telemetry surface from which evaluation request, decision, and decision-relevant input can be reconstructed for the unified audit trail (RFC 0140 §9). Off-engine audit construction is acceptable provided the data is reconstructable.

A PaC engine that does not satisfy (a)–(e) is not CCPE-B conformant in this revision. Engine maintainers whose engine fails one of the criteria are invited to engage via §10; criteria refinements that accommodate engines without weakening the conformance contract are welcomed.

Note the asymmetry with RFC 0140 §4.3 (AGT-class conformance): CCPE-B does not require engine-native trust scoring, execution privilege rings, kill switch, or capability-model enforcement. Those capabilities, where required by the deployment, are supplied by the credential layer or the Gimel Auth Platform per §5.3 — they are not part of the engine-conformance contract.

6. Multi-Tenancy and Policy-Isolation Models

6.1 OPA: Bundle-per-Tenant

OPA's typical multi-tenancy primitive is the bundle: a signed set of policy and data files distributed by the bundle server. Per-tenant isolation is achieved by distributing one bundle per tenant or by partitioning a shared bundle by tenant key. CCPE-B integration: the Phase 2 PaC Adapter supplies the tenant identifier as part of the trust-state PIP input; the OPA policy keys on the tenant identifier to select the relevant policy subtree.

6.2 Cedar: Policy-Store-per-Tenant

Cedar's typical multi-tenancy primitive is the policy store. Per-tenant isolation is achieved by maintaining one policy store per tenant. CCPE-B integration: the Phase 2 PaC Adapter routes the evaluation request to the per-tenant Cedar policy store, with the tenant identifier sourced from the trust-state PIP input.

6.3 SpiceDB: Relationship-Namespace-per-Tenant

SpiceDB's typical multi-tenancy primitive is the relationship-namespace partitioning of the relationship database. Per-tenant isolation is achieved by namespacing the relationship tuples per tenant. CCPE-B integration: the Phase 2 PaC Adapter scopes the permission check to the per-tenant namespace.

6.4 GAuth Credential-Native Multi-Tenancy

GAuth's multi-tenancy primitive is the credential itself. Each agent presents its own PoA credential; the Power-PEP enforces that credential's specific scope, constraints, governance profile, and delegation chain. There is no shared policy store to contaminate; multi-tenancy is structurally guaranteed by the per-credential evaluation model.

6.5 The Chaining Pattern

In CCPE-B, the credential layer's per-credential native multi-tenancy chains to the PaC engine's per-tenant isolation primitive without redundancy. The chain reads:

1. The agent presents its PoA credential. The Power-PEP evaluates Phase 1 against the credential's scope and constraints. (Multi-tenancy here is per-credential.)
2. If Phase 1 PERMITs and Phase 2 is invoked, the Phase 2 PaC Adapter extracts the tenant identifier from the credential's authorization context (typically `mandate.principal.tenant`) and routes to the appropriate per-tenant PaC isolation unit (bundle, policy store, namespace). (Multi-tenancy here is per-tenant.)
3. The PaC engine evaluates Phase 2 against the per-tenant policy. (Multi-tenancy here is per-tenant, scoped within the credential's authorization context.)

The pattern preserves both isolation primitives without conflict: the credential bounds *what authority the agent has*; the per-tenant PaC isolation bounds *what platform policy applies*. Authority and policy compose without one absorbing the other.

7. Comparative Analysis: CCPE-A vs. CCPE-B

This section compares the two CCPE profiles using the nine criteria from GiFo-RFC 0130 §5.2, plus an additional EC-10 row covering Phase-2 CONSTRAIN authority. The comparison is structural: it identifies what stays the same across the two profiles (the credential layer) and what differs (the Phase 2 engine pattern). Both profiles share GAuth's Phase 1 unchanged; differences are entirely in Phase 2.

| Criterion | CCPE-A (RFC 0140 / AGT-class) | CCPE-B (this RFC / PaC-class) | Structural Note |

|---|---|---|---|

| **EC-01 Enforcement Binding** | Hybrid: Phase 1 credential-bound + Phase 2 platform-bound, with engine-native trust state contributing to Phase 2 | Hybrid: Phase 1 credential-bound + Phase 2 platform-bound, with externally supplied trust state contributing to Phase 2 | Identical Phase 1; Phase 2 trust-state ownership differs. |

| **EC-02 Policy Location** | Phase 1: inline in credential. Phase 2: external (YAML / Rego / Cedar inside AGT engine) | Phase 1: inline in credential. Phase 2: external (Rego bundle, Cedar policy store, SpiceDB relationship database) | Both profiles externalize Phase 2 policy; the storage primitive differs. |

| **EC-03 Granularity** | Phase 1: verb-resource-constraint. Phase 2: tool-call (AGT-class binary ALLOW/DENY) | Phase 1: verb-resource-constraint. Phase 2: engine-native (Rego arbitrary structured; Cedar entity-typed; SpiceDB relationship-permission) | CCPE-B can express finer-grained Phase 2 conditions where the policy author chooses to. Cedar deployments are an exception (see §4.3 limitation callout): Cedar is binary by design and cannot carry structured Phase-2 obligations through the Decision Envelope. |

| **EC-04 Multi-Tenant Model** | Phase 1: per-credential native. Phase 2: shared AGT instance with policy branching | Phase 1: per-credential native. Phase 2: per-tenant bundle / policy store / namespace | CCPE-B's Phase 2 multi-tenancy is engine-native rather than configuration-native. |

| **EC-05 Multi-Hop Delegation** | Phase 1: built-in (CHK-16). Phase 2: not addressed (AGT does not verify delegation hops). | Phase 1: built-in (CHK-16). Phase 2: not addressed (PaC engines do not verify delegation hops). | Identical. Multi-hop

verification is a Phase 1 property; Phase 2 in both profiles inherits the Phase 1 result. |

| **EC-06 Human Oversight** | Phase 1: mandatory one-time structural gate. Phase 2: optional per-action AGT prompt | Phase 1: mandatory one-time structural gate. Phase 2: deployment-defined (PaC engines do not natively offer per-action HITL) | CCPE-B Phase 2 HITL, where present, is implemented at the deployment layer outside the engine. |

| **EC-07 Interoperability** | Standards-based at credential layer (RFC 0116). AGT-class engine pattern is documented in RFC 0140. | Standards-based at credential layer (RFC 0116). PaC integration profile is this RFC; engine policy languages are governed by the engines' own community processes. | Both profiles are interoperable at the credential layer; CCPE-B inherits the PaC ecosystem's policy-language interoperability properties. |

| **EC-08 Threat Detection** | Phase 1: GAuth deterministic open corpus (RFC 0117). Phase 2: AGT semantic intent classifier (Type C, proprietary per RFC 0140 §8) | Phase 1: GAuth deterministic open corpus. Phase 2: engine-native (Rego/Cedar/SpiceDB do not ship a threat detection corpus; deployment-supplied). | CCPE-B Phase 2 threat detection is the deployment's responsibility. The Gimel Auth Platform optionally exposes the GAuth threat-detection corpus as a PIP input the PaC policy can reference. |

| **EC-09 Regulatory Alignment** | Phase 1: GDPR, NIS2, DORA, EU AI Act, BSI; broad coverage of OWASP ASI Top 10 and OWASP LLM Top 10 (per-item mapping ledger to be published with revision 1.0). Phase 2: AGT vendor-claimed mapping. | Phase 1: identical. Phase 2: engine-dependent; PaC engines do not generally claim regulatory mappings. | Phase 1 regulatory coverage is identical across CCPE-A and CCPE-B. |

| **EC-10 Phase-2 CONSTRAIN Authority** | Phase 2 (AGT) does **not** produce or modify CONSTRAIN; CONSTRAIN is issued exclusively by Phase 1 Power-PEP, optionally consuming Phase-2 obligations as inputs (RFC 0140 §2.1). | Phase 2 (PaC) does **not** produce or modify CONSTRAIN; CONSTRAIN is issued exclusively by Phase 1 Power-PEP, optionally consuming Phase-2 obligations as inputs (this RFC §4.1). | **Identical across CCPE-A and CCPE-B.** Keeping CONSTRAIN issuance in Phase 1 across both profiles preserves the credential-bound layer as the sole authority for narrowing the agent's scope. |

The single structural property worth naming directly: what changes across CCPE-A and CCPE-B is exclusively Phase 2. Phase 1 - the credential-bound layer where authority is enforced, delegation chains verified, governance profiles applied, and budgets tracked - is identical across the two profiles. CCPE-A and CCPE-B are not competing approaches to AI governance; they are two valid Phase 2 instantiations of the same credential-bound architecture, chosen according to the deployment's existing platform-policy infrastructure.

8. Security Considerations

Status of the integration profile. This RFC specifies an integration architecture; it does not assess the security posture of any specific engine or any specific deployment. A CCPE-B deployment's security posture is the joint property of (a) GAuth's Phase 1 properties as specified in RFCs 0110–0118, (b) the PaC engine's documented properties, (c) the trust-state PIP feeds' integrity and freshness, and (d) the deployment's operational discipline.

PaC bundle / policy store / relationship database supply chain. Each exemplar engine has a published supply-chain model (OPA bundle signing, Cedar policy versioning, SpiceDB schema migration). Compromise of the policy distribution channel would allow an attacker to substitute Phase 2 policies that PERMIT what the legitimate policy would DENY. CCPE-B does not weaken these supply-chain properties and does not introduce new ones; deployments Should follow the engine maintainer's published supply-chain guidance and Should additionally verify, at the unified audit layer (RFC 0140 §9), that the bundle / store / namespace version evaluated for each decision matches the version expected for the deployment's release manifest.

Trust-state PIP freshness and replay. The trust-state object supplied per evaluation (§5.3) carries fields that can change between evaluations (`agent.trust_score`, `agent.kill_switch_state`, `agent.circuit_breaker_state`, `mandate.budget_remaining`). The Phase 2 PaC Adapter Must source these from the authoritative producers (GAuth and the Gimel Auth Platform) at evaluation time, not from cached values. Caching the trust-state object across evaluations risks a replay where a policy evaluates against stale kill-switch state and PERMITs an action the current state would DENY. Implementations Should additionally bind the trust-state object to a short-lived nonce or timestamp that the Power-PEP verifies at the combined-verdict step.

Phase 1 attestation reuse (Embedded topology). In the CCPE-B / Embedded subprofile (§4.2.2), the Power-PEP issues a Phase 1 attestation that downstream applications consume as PIP input to their local PaC evaluation. The attestation's required and optional claims are normatively specified in §4.2.2.1. The cryptographic binding requirements are:

- PoA binding. The `cnf` claim Must include both a reference to the PoA credential (`poa_ref`) and a thumbprint over a canonicalized form of the PoA credential (`poa_thumbprint`, base64url SHA-256 over the JCS-canonicalized PoA per RFC 8785). Downstream consumers Must verify that the active credential matches the thumbprint before accepting the attestation.
- Action-class binding. The `gauth_action` claim Must contain the verb-resource-constraint Phase 1 evaluated. Downstream consumers Must reject any attestation whose `gauth_action` does not match the action they are about to authorize.
- TTL binding. The `exp` claim Must Not exceed 60 seconds beyond `iat` for routine attestations. Longer-lived attestations are permitted only for explicit batch operations and require a deployment-published TTL policy.
- Audience binding. The `aud` claim Must identify the authorized downstream consumer(s). Single-audience attestations are Recommended. Downstream consumers Must reject any attestation whose `aud` does not include their own identifier.
- Signing key. Attestations Must be signed using EdDSA (Ed25519, RFC 8032) or RS256 / ES256 per RFC 7518. The signing key Must be published at a stable JWKS URI under the GAuth deployment's authoritative URL and Must be rotated per the deployment's key-management policy. The `jti` claim Must be unique per attestation; downstream consumers Should maintain a short-lived `jti`-replay cache sized to the attestation TTL.

Without these bindings, an attestation could be replayed against a different action class, against a different downstream application, or after the underlying mandate's revocation - converting a Phase-1 PERMIT for action A against application X into apparent authority for action B against application Y.

Power*Point ↔ Policy*Point chain failure modes. The chain has two evaluators and therefore two failure modes in addition to those documented for each evaluator separately:

```
| Failure mode | Phase 1 outcome | Phase 2 outcome | Default
combined verdict (`phase2_required = true`) | With opt-in
availability posture (`phase2_required = false`) |

|---|---|---|---|---|

| Both healthy | PERMIT | PERMIT | PERMIT | PERMIT |

| Both healthy, Phase 2 deny | PERMIT | DENY | DENY | DENY |

| Phase 1 deny | DENY | (not invoked) | DENY | DENY |

| Phase 1 healthy, Phase 2 degraded (engine unreachable,
bundle corruption, schema migration in flight) | PERMIT |
```

```
(failure) | **DENY** — fail-closed default, aligned with RFC
0140 v1.2 §11.5 | **PERMIT** — opt-in fall-back to Phase 1
verdict (dual-impulse), see prose below |

| Phase 1 degraded (GAuth unreachable, credential cannot be
validated) | (failure) | (any) | DENY | DENY |
```

The default behavior on Phase-2 degradation is fail-closed (DENY), in alignment with RFC 0140 v1.2 §11.5 ("If Phase 2 is unavailable, the result Must be DENY"). The fail-closed default reflects that Phase 2 exists precisely because the credential-bound Phase-1 layer was not necessarily deemed sufficient for the action class in question; falling back to Phase 1 silently undoes that deployment decision. Deployments whose Phase 2 is purely defense-in-depth on top of an already-regulatory-sufficient Phase 1 — i.e., where the credential-bound layer is the authoritative regulatory enforcement and Phase 2 only adds platform-specific risk gating — May set `phase2_required = false` per mandate to accept the Phase-1 verdict during transient Phase-2 unavailability ("dual-impulse" availability posture). This setting is an opt-in posture choice and Should be documented in the deployment's risk register. The flag is carried as a constraint object on the PoA credential, surfaced in the Phase 1 attestation per §4.2.2.1, and consulted by the Phase 2 PaC Adapter at fall-back decision time. The flag's absence Must be treated as `true` (fail-closed).

The Phase-1 degraded case is unconditional DENY irrespective of `phase2_required`: PaC engines have no basis for evaluating delegated authority; absence of Phase 1 is absence of authority, not presence of authority.

Alignment with RFC 0140 v1.2 §11.5. RFC 0140 v1.2 §11.5 ("Fail-Closed Assurance") states normatively: "G-AGT implementations Must never fall back to Phase 1-only evaluation. If Phase 2 is unavailable, the result Must be DENY." CCPE-B's default behavior in the Phase-1-healthy / Phase-2-degraded row applies the same principle: a Phase-2 evaluator that cannot produce a verdict produces a combined DENY by default. CCPE-B additionally surfaces a per-mandate `phase2_required = false` opt-in for deployments whose risk model treats Phase 2 as defense-in-depth on top of an already-regulatory-sufficient Phase 1; that opt-in is an explicit deployment posture choice rather than the default. In both profiles, Phase-1 degradation is unconditional DENY.

Multi-engine consistency in Hybrid topology. Where a deployment routes different action classes to different Phase 2 evaluators (some to a centralized OPA, some to embedded Cedar embedments), the routing rule itself becomes a security-relevant configuration. Misconfiguration could route an action class to an evaluator with a more permissive policy than the deployment's intended policy. The unified audit trail (RFC 0140 §9) Should record which Phase 2 evaluator handled each decision so that routing-rule errors are detectable post-hoc.

Note on out-of-scope detection (RFC 0130 §12 reference). The lagging-indicator property of behavioural trust scores documented in GiFo-RFC 0130 §12 applies in CCPE-B insofar as a deployment chooses to feed a trust score (sourced from the Gimel Auth Platform per §5.3) into PaC policy evaluation. The same caveats apply: a behavioral trust score is a lagging indicator of known-pattern violations, not a leading indicator of novel threats.

9. References

A. GiFo Internal Cross-References (Normative)

- GiFo-RFC 0080 — Gimel Foundation Legal Provisions Relating to GiFo Documents.
- GiFo-RFC 0090 — Gimel Foundation Rights in Contributions.
- GiFo-RFC 0100 — Gimel Foundation Intellectual Property Rights Policy.
- GiFo-RFC 0110 — GAuth Protocol Engine (Power*Point architectural pattern; PEP/PDP/PAP/PIP/PVP).
- GiFo-RFC 0111 — GAuth Authorization Framework.
- GiFo-RFC 0115 — Power-of-Attorney Credential Definition.
- GiFo-RFC 0116 — GAuth Interoperability (Extended Token; W3C VC representation; OAuth engine integration).
- GiFo-RFC 0117 — GAuth PEP Interface (16-check enforcement pipeline; Phase 2 extension point this RFC integrates into).
- GiFo-RFC 0118 — GAuth Management API.
- GiFo-RFC 0130 — AI Governance Tool Evaluation (PBRG category definition; nine-criterion framework reused in §7).
- GiFo-RFC 0140 — G-AGT Integration Profile (CCPE-A); v1.2 published.

B. PaC Ecosystem Sources (Primary)

- Open Policy Agent / Rego - OPA project documentation (CNCF graduated). Rego policy language reference. Bundle server protocol.
- Cedar - Cedar policy language specification (AWS / open source). Cedar evaluation semantics.
- Zanzibar / SpiceDB - Authzed SpiceDB documentation. Zanzibar pattern (Google research paper, "Zanzibar: Google's Consistent, Global Authorization System").
- OpenFGA - OpenFGA project documentation (CNCF, Zanzibar-pattern; out-of-scope for CCPE-B v0.9 but architecturally analogous to SpiceDB).
- Permit.io, Styra - commercial OPA-based authorization platforms; vendor product documentation (additional engines welcomed via §10).

- IETF RFC 2753 - "A Framework for Policy-based Admission Control" (origin of Policy*Point terminology).
- OASIS XACML 3.0 - eXtensible Access Control Markup Language (PEP/PDP/PAP/PIP decomposition for access control).
- IETF RFC 7519 - JSON Web Token (JWT) — used for the Phase 1 attestation in §4.2.2.1.
- IETF RFC 7800 - Proof-of-Possession Key Semantics for JSON Web Tokens — `cnf` claim semantics.
- IETF RFC 8032 - Edwards-Curve Digital Signature Algorithm (EdDSA) — Ed25519 signing for attestations.
- IETF RFC 8392 - CBOR Web Token (CWT) — alternative attestation envelope per §4.2.2.1.
- IETF RFC 8785 - JSON Canonicalization Scheme (JCS) — used for PoA thumbprinting in `cnf` per §8.

C. Industry Validation Sources (Third-Party, Neutral)

- Sondera / Maisel, Matt — "Hooking Coding Agents with the Cedar Policy Language" (blog post + conference talk).
- EQTY Lab, with Trail of Bits — Cupcake (OPA / Rego policy framework for agent actions).
- OpenClaw — open-source policy-aware agent loop with Cedar integration.
- Windley, Phil — "A Policy-Aware Agent Loop with Cedar and OpenClaw".
- Hughes, Chris — "A Look At An Emerging Runtime Enforcement Layer For Agents — Hooks". Resilient Cyber, April 13, 2026 (background industry-convergence framing inherited from RFC 0130 §3).

D. Per-Engine Citation Ledger (Back-Reference to Appendix A)

The per-engine citation ledger - one row per engine-behaviour claim made in §3.1, §4.3, §5.1–5.2, §6, and §7, with source URL, access date, and verbatim quoted evidence - is published in Appendix A of this document. Appendix A delivers what earlier drafts of this section deferred to revision 1.0: the engine documentation URL, policy-language specification URL, multi-tenancy primitive documentation URL, and last-accessed date for each in-scope engine. Engine maintainers reviewing through §10 should consult Appendix A first to verify which source URLs and which paraphrased summaries underlie the claims in the normative sections, and to flag any that mischaracterize their engine's documented behaviour.

- Open Policy Agent (OPA / Rego) — CNCF
- Cedar — AWS / open source

- SpiceDB — Authzed
- (additional engines added on engine-maintainer request)

10. How to Suggest a Correction

This RFC is published as a work-in-progress draft. Corrections, clarifications, and additions are actively solicited.

Channels:

- GitHub Issues - open an issue at `https://github.com/Gimel-Foundation/gifo-rfc-0150` with the label `engine-review` (for engine maintainers and contributor implementations named in §3) or `correction` (for general corrections). Include the section reference and the source you would like cited.
- Email - send corrections to `info@gimelfoundation.com`. Include the section reference, the proposed correction, and a citation source. Engine self-descriptions sourced from the engine's own published materials can be incorporated verbatim with attribution.
- Pull Request - fork the repository, make the change, and open a PR against the `engine-review` branch.

11. Acknowledgments

This RFC builds on the architectural pattern surfaced by GiFo-RFC 0140 §3.8.1 ("Policy-as-Code Engines", under §3.8 Engine Pattern Extensibility) and on the contributor implementations surveyed in GiFo-RFC 0130 §4.5. The Power*Point ↔ Policy*Point distinction that grounds the integration semantics is rooted in the deliberate terminological choice of GiFo-RFCs 0110 and 0111.

The named engine families - Open Policy Agent (CNCF), Cedar (AWS / open source), and Zanzibar / SpiceDB (Authzed, after Google's published Zanzibar work) - and their respective communities are acknowledged as the authoritative stewards of the PaC architecture this RFC integrates with. CCPE-B does not modify any engine's policy language, evaluation semantics, or operational model; it specifies how GAuth's credential-bound enforcement chains *to* the engines as documented by their maintainers.

The contributor implementations named in §3.2 - Sondera (Maisel), Cupcake (EQTY Lab with Trail of Bits), OpenClaw, and Phil Windley - are acknowledged as the prior reference surface that demonstrated PaC engines driving agent hooks. CCPE-B specifies the credential-layer extension those implementations would build on to gain credential-bound authority enforcement.

Engine maintainers and contributor-implementation authors who wish to be acknowledged by name in revision 1.0 are invited to engage via §10.

* * *

Appendix A — Per-Engine Citation Ledger (Informative)

This appendix back-cites the engine-behaviour claims made in §3.1, §4.2.2.1, §4.3, §5.1–5.2, §6, §7, and §8 to publicly available primary sources. Each row captures: the section in which the claim appears, the claim text (paraphrased), the source URL where the corresponding documentation lives, the date the source was consulted, the reference release of the source at the access date, and a short evidence note quoting the supporting passage verbatim.

Deliverable form. Per editorial direction at v0.9.5, the per-engine citation ledger is embedded inline as Appendix A of RFC 0150 (informative). The standalone companion file `GiFo_RFC_0150_Appendix_A_Citation_Ledger.md` previously published alongside earlier revisions is retired as of v0.9.5; engine maintainers seeking to review the ledger as an independent artifact are referred to this appendix and may extract it through the §10 correction channel. The §9.D References note back-references this appendix so readers locating the spec via §9 can find the ledger.

Status of evidence column. The "Quoted evidence" column carries verbatim text from each source as observed at the access date noted, ≤ 2 sentences per row. Where wording in the live source has drifted since the access date, engine maintainers reviewing through §10 may submit the canonical replacement quote. Where the source is HTML documentation that does not have a stable canonical sentence (e.g., concept overviews assembled from headings + lists), the row uses the source's own headline statement or the most stable single sentence found at the URL. Existing engine-behavior rows use the access date 2026-04-18 (carried forward from v0.9.1, the last revision in which engine documentation was systematically re-verified). Cross-cutting IETF references added at v0.9.4 (RFC 7519 / 7800 / 8032 / 8392 / 8785) use the access date 2026-04-19. Future revisions may refresh access dates en bloc.

Status of negative claims (best-evidence hypotheses). Rows that assert the *absence* of a capability from an engine's documented feature surface (§5.2 / §7 EC-05, EC-06, EC-08, EC-09) cite the engine's own published scope statement as the best available primary source. A scope statement establishes what the engine *is*; the absence of a capability from the engine's documentation is the best-evidence basis for the negative claim, but it is not equivalent to a positive vendor disclaimer. These rows are therefore marked as **best-evidence hypotheses pending §10 maintainer confirmation**. Engine maintainers may, through §10, either (a) confirm the absence with a vendor statement that becomes the canonical citation, or (b) provide the documentation of a capability the absence claim asserts and the row is then withdrawn / restated.

Status of release column. The "Reference release" column lists the latest released version observed at the access date together with the canonical release page URL where the current version may be verified. Engine maintainers may update the version field via §10; the release URL is the authoritative source.

Coverage discipline (1:1 mapping). Each row maps to exactly one engine-behavior claim made in the cited section. Where a single ledger row is reused as evidence for multiple sections (e.g., an OPA-deployment claim cited by both §3.1 and §6.1), the reuse is shown by listing both section anchors in the § column. Where a section makes a claim about an engine that is *not* yet covered by a ledger row, that gap is logged as an action item under §A.6 below for resolution in the next revision.

Coverage in this revision. This v0.9.5 ledger covers the three OSS engines named in §3.1 (OPA / Rego, Cedar, SpiceDB), the cross-cutting policy / IETF references (XACML, RFC 2753, RFC 2119, RFC 7519, RFC 7800, RFC 8032, RFC 8392, RFC 8785), and the architectural-analogy entries for adjacent engines (OpenFGA, Permit.io, Styra DAS, Casbin). Per-claim rows for the adjacent commercial / managed engines remain reserved pending §10 maintainer engagement.

A.1 Open Policy Agent (OPA) / Rego — CNCF graduated project

| § | Claim | Source URL | Access | Reference release | Quoted evidence (verbatim) |

|---|---|---|---|---|---|

| 3.1 | OPA is hosted by the CNCF as an open source, general-purpose policy engine (graduated maturity level per CNCF project listing). | https://www.cncf.io/projects/open-policy-agent-opa/ | 2026-04-18 | OPA: Graduated maturity level on CNCF project listing. Release page: https://github.com/open-policy-agent/opa/releases | "Open Policy Agent (OPA) is an open source, general-purpose policy engine that unifies policy enforcement across the stack." |

| 3.1 | Rego is OPA's declarative policy language. | https://www.openpolicyagent.org/docs/latest/policy-language/ | 2026-04-18 | OPA v1.x current series. Release page: https://github.com/open-policy-agent/opa/releases | "OPA policies are expressed in a high-level declarative language called Rego." |

| 3.1 / 6.1 | OPA accepts arbitrary structured input and decouples decision-making from enforcement; deployment patterns include sidecar, library, and daemon. | https://www.openpolicyagent.org/docs/latest/ | 2026-04-18 | OPA v1.x current series. Release page: https://github.com/open-policy-agent/opa/releases | "OPA decouples policy decision-making from policy enforcement. When

your software needs to make policy decisions it queries OPA and supplies structured data (e.g., JSON) as input." |

| 3.1 / 6.1 | OPA bundles are the primary primitive for distributing policy and data, supporting per-tenant partitioning. | https://www.openpolicyagent.org/docs/latest/management-bundles/ | 2026-04-18 | OPA v1.x current series. Release page: https://github.com/open-policy-agent/opa/releases | "OPA can periodically download bundles of policy and data from remote HTTP servers. The policies and data are loaded on the fly without requiring a restart of OPA." |

| 4.3 | OPA returns structured Rego results whose document shape is policy-author-controlled. | https://www.openpolicyagent.org/docs/latest/policy-language/ | 2026-04-18 | OPA v1.x current series | "Rules define the content of documents. Documents can be defined solely by rules or be defined by both rules and data loaded from outside of OPA." |

| 5.1 | OPA evaluation is a deterministic function of policy and input. | https://www.openpolicyagent.org/docs/latest/philosophy/ | 2026-04-18 | OPA v1.x current series | "OPA decouples policy decisions from the software that needs them so that you can change policies without recompiling your code or services. With OPA, you can write policy in a declarative language called Rego that is purpose-built for expressing policies over complex hierarchical data structures." |

| 5.1 | OPA exposes a decision-log interface for capturing evaluation events. | https://www.openpolicyagent.org/docs/latest/management-decision-logs/ | 2026-04-18 | OPA v1.x current series | "OPA can periodically report decision logs to remote HTTP servers, using custom plugins, or to the console output." |

| 5.2 / 7 (EC-05, EC-06, EC-08, EC-09) — *best-evidence hypothesis* | OPA's published self-description scopes it as a policy engine for unifying enforcement; trust scoring, privilege rings, kill switches, circuit breakers, mandate state, budgets, delegation-hop verification, per-action human-in-the-loop, threat-detection corpus, and regulatory mappings are not surfaced in its documented feature set. *Pending §10 maintainer confirmation.* | https://www.openpolicyagent.org/ | 2026-04-18 | OPA v1.x current series | "Policy-based control

for cloud native environments. Flexible, fine-grained control for administrators across the stack." |

A.2 Cedar — open source under Apache 2.0; originated by AWS

| § | Claim | Source URL | Access | Reference release | Quoted evidence (verbatim) |

|---|---|---|---|---|---|

| 3.1 | Cedar is an open-source policy language for expressing fine-grained authorization. | https://www.cedarpolicy.com/ | 2026-04-18 | Cedar current release series. Release page: https://github.com/cedar-policy/cedar/releases | "Cedar is an open-source language for writing authorization policies and making authorization decisions based on those policies." |

| 3.1 | Cedar policies are strongly typed and validated against a schema. | https://docs.cedarpolicy.com/schema/schema.html | 2026-04-18 | Cedar current release series. Release page: https://github.com/cedar-policy/cedar/releases | "Cedar policies and entities can be validated against a schema. A schema is a declaration of the structure of the entity types that you want to support in your application, along with the actions that you want users to perform on them." |

| 3.1 | Cedar evaluation returns Allow or Deny with diagnostics including the satisfying policies. | https://docs.cedarpolicy.com/auth/authorization.html | 2026-04-18 | Cedar current release series | "An authorization request asks the question, 'Can this principal take this action on this resource in this context?' The Cedar authorization engine evaluates the request against the policies and entities you provide, and returns an authorization response that includes a decision — Allow or Deny — and a list of the policies that contributed to the decision." |

| 3.1 | Cedar is library-embedded in applications; policy stores hold the active policy set. | https://docs.cedarpolicy.com/overview/intro.html | 2026-04-18 | Cedar current release series | "You can use Cedar in your application by integrating the open source Cedar policy language and authorization engine." |

| 4.3 / 6.2 | Cedar's per-tenant isolation is achieved via a per-tenant policy store. | https://docs.cedarpolicy.com/policies/policy-store.html | 2026-04-18 | Cedar current release series | "A policy store is a logical container for the Cedar policies, schema, and policy templates that you use to authorize access to your application's resources." |

| 5.1 | Cedar evaluation is deterministic and verification-guided. | https://www.cedarpolicy.com/ | 2026-04-18 | Cedar current release series | "Cedar's design and implementation use verification-guided development to provide high assurance: we use formal methods to model Cedar and prove key correctness properties about it." |

| 5.1 | Cedar evaluation returns diagnostics enumerating the policies that contributed to the decision (audit-logging surface). | https://docs.cedarpolicy.com/auth/authorization.html | 2026-04-18 | Cedar current release series | "The response includes a decision — Allow or Deny — and a list of the policies that contributed to the decision." |

| 5.2 / 7 (EC-05, EC-06, EC-08, EC-09) — *best-evidence hypothesis* | Cedar's published self-description scopes it as an authorization-policy engine; trust scoring, privilege rings, kill switches, circuit breakers, mandate state, budgets, delegation-hop verification, per-action human-in-the-loop, threat-detection corpus, and regulatory mappings are not surfaced in its documented feature set. *Pending §10 maintainer confirmation.* | https://www.cedarpolicy.com/ | 2026-04-18 | Cedar current release series | "Cedar is an open-source language for writing authorization policies and making authorization decisions based on those policies." |

A.3 Zanzibar / SpiceDB — Authzed; pattern after Google Zanzibar

| § | Claim | Source URL | Access | Reference release | Quoted evidence (verbatim) |

|---|---|---|---|---|---|

| 3.1 | SpiceDB is the open-source authorization database from Authzed implementing the Zanzibar pattern. | https://authzed.com/docs/spicedb/getting-started/discovering-spicedb | 2026-04-18 | SpiceDB v1.x current series. Release page: https://github.com/authzed/spicedb/releases | "SpiceDB

is an open source, Google Zanzibar-inspired database system for creating and managing security-critical application permissions." |

| 3.1 | The Zanzibar pattern is documented in Google's published research. | https://research.google/pubs/zanzibar-googles-consistent-global-authorization-system/ | 2026-04-18 | Academic paper; USENIX ATC '19 | "Determining whether online users are authorized to access digital objects is central to preserving privacy. This paper presents the design, implementation, and deployment of Zanzibar, a global system for storing and evaluating access control lists." |

| 3.1 | SpiceDB models authorization as a schema plus relationships; permission checks traverse the relationship graph. | https://authzed.com/docs/spicedb/concepts/schema | 2026-04-18 | SpiceDB v1.x current series | "A schema defines the objects, the relationships those objects can have to one another, and the permissions that can be computed off of those relationships." |

| 3.1 | SpiceDB is deployed as a networked authorization service backed by a datastore. | https://authzed.com/docs/spicedb/getting-started/installing-spicedb | 2026-04-18 | SpiceDB v1.x current series | "SpiceDB is a database that can be used to store, compute, and validate your application's permissions." |

| 4.3 | SpiceDB exposes a CheckPermission API returning a boolean check result. | https://authzed.com/docs/spicedb/getting-started/protecting-a-blog | 2026-04-18 | SpiceDB v1.x current series | "Once we've written some relationships, we can ask SpiceDB whether a particular subject has permission to perform an action on a particular resource using the CheckPermission API." |

| 4.3 | SpiceDB also exposes an ExpandPermissionTree API returning the permission expansion (tree of subjects contributing to the permission). | https://authzed.com/docs/spicedb/getting-started/faq | 2026-04-18 | SpiceDB v1.x current series. API reference: https://buf.build/authzed/api/docs/main:authzed.api.v1#authzed.api.v1.PermissionsService.ExpandPermissionTree | "ExpandPermissionTree reveals the relationships that grant access to a permission for a given resource." |

| 5.1 | SpiceDB exposes a Watch API for streaming relationship-update events (audit-logging surface for

relationship changes). |
https://authzed.com/docs/spicedb/concepts/watch-api | 2026-04-18 | SpiceDB v1.x current series | "The Watch API in SpiceDB allows you to subscribe to a stream of all changes occurring in a SpiceDB cluster as they happen." |

| 6.3 | SpiceDB supports per-tenant isolation via object-definition / namespace partitioning in the schema. | https://authzed.com/docs/spicedb/concepts/schema | 2026-04-18 | SpiceDB v1.x current series | "Definitions are the basic building blocks of a schema. They define the types of objects that exist in the system, along with the relationships and permissions that apply to those objects." |

| 5.2 / 7 (EC-05, EC-06, EC-08, EC-09) — *best-evidence hypothesis* | SpiceDB's published self-description scopes it as a permissions database; trust scoring, privilege rings, kill switches, circuit breakers, mandate state, budgets, delegation-hop verification, per-action human-in-the-loop, threat-detection corpus, and regulatory mappings are not surfaced in its documented feature set. *Pending §10 maintainer confirmation.* |
https://authzed.com/docs/spicedb/getting-started/discovering-spicedb | 2026-04-18 | SpiceDB v1.x current series | "SpiceDB is an open source, Google Zanzibar-inspired database system for creating and managing security-critical application permissions." |

A.4 Cross-Cutting References

| § | Claim | Source URL | Access | Reference release | Quoted evidence (verbatim) |

|---|---|---|---|---|---|

| 2 / Notational Conventions | XACML defines the Policy*Point decomposition (PEP, PDP, PAP, PIP). | https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html | 2026-04-18 | OASIS Standard, XACML v3.0, 22 January 2013 | "The major actors in the XACML domain" are defined as the PAP (creates policy), PDP (evaluates policy and renders decisions), PEP (performs access control by making decision requests and enforcing decisions), and PIP (source of attribute values). |

| Notational Conventions | IETF RFC 2753 frames the policy-based admission control architecture from which the Policy*Point lineage descends. |

https://datatracker.ietf.org/doc/html/rfc2753 | 2026-04-18 | IETF Informational RFC, January 2000 | "This document is concerned with specifying a framework for providing policy-based control over admission control decisions … Two main architectural elements are defined for policy control: the Policy Enforcement Point (PEP) and the Policy Decision Point (PDP)." |

| Notational Conventions | RFC 2119 defines the requirement-level keywords used in this document. | https://datatracker.ietf.org/doc/html/rfc2119 | 2026-04-18 | IETF Best Current Practice 14, March 1997 | "In many standards track documents several words are used to signify the requirements in the specification. These words are often capitalized. This document defines these words as they should be interpreted in IETF documents." |

| 4.2.2.1 | IETF RFC 7519 (JSON Web Token) defines the envelope format used for the Phase 1 attestation. | https://datatracker.ietf.org/doc/html/rfc7519 | 2026-04-19 | IETF Proposed Standard, May 2015 | "JSON Web Token (JWT) is a compact, URL-safe means of representing claims to be transferred between two parties. The claims in a JWT are encoded as a JSON object that is used as the payload of a JSON Web Signature (JWS) structure or as the plaintext of a JSON Web Encryption (JWE) structure, enabling the claims to be digitally signed or integrity protected with a Message Authentication Code (MAC) and/or encrypted." |

| 4.2.2.1 / 8 | IETF RFC 7800 (Proof-of-Possession Key Semantics for JWTs) defines the `cnf` claim used to bind the Phase 1 attestation to the underlying PoA credential. | https://datatracker.ietf.org/doc/html/rfc7800 | 2026-04-19 | IETF Proposed Standard, April 2016 | "This specification describes how to declare in a JSON Web Token (JWT) that the presenter of the JWT possesses a particular proof-of-possession key and how the recipient can cryptographically confirm proof of possession of the key by the presenter." |

| 4.2.2.1 / 8 | IETF RFC 8032 (Edwards-Curve Digital Signature Algorithm, EdDSA) defines the Ed25519 signing scheme recommended for Phase 1 attestation signatures. | https://datatracker.ietf.org/doc/html/rfc8032 | 2026-04-19 | IETF Informational RFC, January 2017 | "This document describes elliptic curve signature scheme Edwards-curve Digital Signature Algorithm (EdDSA). The algorithm is

instantiated with recommended parameters for the edwards25519 and edwards448 curves." |

| 4.2.2.1 | IETF RFC 8392 (CBOR Web Token, CWT) defines the alternative attestation envelope permitted for resource-constrained deployments. | https://datatracker.ietf.org/doc/html/rfc8392 | 2026-04-19 | IETF Proposed Standard, May 2018 | "CBOR Web Token (CWT) is a compact means of representing claims to be transferred between two parties. The claims in a CWT are encoded in the Concise Binary Object Representation (CBOR), and CBOR Object Signing and Encryption (COSE) is used for added application-layer security protection." |

| 8 | IETF RFC 8785 (JSON Canonicalization Scheme, JCS) defines the canonicalization used for the PoA thumbprint carried in the `cnf` claim. | https://datatracker.ietf.org/doc/html/rfc8785 | 2026-04-19 | IETF Informational RFC, June 2020 | "Cryptographic operations like hashing and signing depend on the data to be processed being expressed in an invariant format … JSON Canonicalization Scheme (JCS) achieves this by combining the strict serialization of JSON data formats with the JSON Number serialization defined by ECMAScript." |

A.5 Adjacent Engines — Canonical Sources Reserved (Per-Claim Rows Pending §10 Engagement)

The following adjacent PaC engines were named in §3.1 as architecturally analogous to the in-scope set. §3.1's only direct claim about each is the analogy itself (commercial / OSS status, OPA-based vs. Zanzibar-pattern); §3.1 does not make per-engine behavior claims that require their own ledger rows. When an engine maintainer engages through §10 (or when a future revision adds per-engine behavior claims to the normative core), the rows are added to the appropriate subsection above and the canonical URL set below becomes the row's Source URL / Reference release seed:

| Engine | Canonical doc URL | Reference release URL | §3.1 architectural claim |

|---|---|---|---|

| **OpenFGA** | https://openfga.dev/docs | https://github.com/openfga/openfga/releases | CNCF; Zanzibar-pattern (analogous to SpiceDB). |

| **Permit.io** | https://docs.permit.io | https://github.com/permitio (multiple component repos) | Commercial OPA-based authorization platform. |

| **Styra DAS** | https://docs.styra.com | Commercial product; release notes published in vendor docs | Commercial OPA-based authorization platform. |

| **Casbin** | https://casbin.org/docs | https://github.com/casbin/casbin/releases | Open-source authorization library with multiple language bindings. |

A.6 Coverage Audit — §3.1, §3.4, §4.2.2.1, §4.3, §4.3.1, §4.4.1, §5.1–5.3, §6, §7, §8

This audit walks the scoped sections claim-by-claim and confirms ledger coverage. Sections that make no engine-specific behavior claim (only architectural framing, GiFo-internal cross-references, or normative GAuth-side specification) are noted as "no engine-behavior claim" and are out of citation-ledger scope by design.

| Section | Engine-behavior claims made | Ledger row(s) | Status |

|---|---|---|---|

| §3.1 (OPA row of engines table) | OPA = CNCF graduated; Rego = declarative policy language; sidecar/library/daemon; bundle-server distribution | A.1 rows 1–4 | Covered. |

| §3.1 (Cedar row of engines table) | Cedar = AWS / open source under Apache 2.0; strongly-typed schema-validated; Allow/Deny + diagnostics; library-embedded with policy stores | A.2 rows 1–4 | Covered. |

| §3.1 (SpiceDB row of engines table) | SpiceDB = Authzed open-source implementing Zanzibar; relationship tuples + schema; networked authorization service | A.3 rows 1–4 | Covered. |

| §3.1 (adjacent engines paragraph) | Permit.io / Styra / OpenFGA / Casbin "share enough architectural shape" to apply CCPE-B | A.5 (architectural-analogy claim only; no per-engine behavior claim made) | Covered as architectural-analogy entries; no behavior claim to back-cite. |

| §3.2 (contributor implementations) | No engine-behavior claim (describes contributor-implementation provenance; covered by §9.C primary sources) | n/a | Out of citation-ledger scope by design. |

| §3.4 (Profile Determination) | No engine-behavior claim (decision-tree about deployment shape and ownership model; references RFC 0140 §4.3 for AGT-class conformance criteria) | n/a | Out of citation-ledger scope by design. |

| §4.2.2.1 (Phase 1 Attestation Format) | No engine-behavior claim; specifies GAuth-side attestation envelope using IETF JWT (RFC 7519), PoP `cnf` (RFC 7800), EdDSA (RFC 8032), CWT (RFC 8392), and JCS canonicalization (RFC 8785) | A.4 rows 4–8 (IETF references) | Covered. |

| §4.3 (engine return shapes) | OPA = arbitrary structured Rego result; Cedar = Allow/Deny + diagnostics; SpiceDB = boolean check + optional permission expansion | A.1 row 5; A.2 row 3; A.3 rows 5–6 (Check + Expand) | Covered. |

| §4.3.1 (mapping Decision Envelope to RFC 0140 §9 audit record) | No engine-behavior claim (specifies the GiFo-internal audit-record mapping; references RFC 0140 §9 unified audit trail) | n/a | Out of citation-ledger scope by design. |

| §4.4.1 (Open-Core Boundary and Type-C Dependency) | No engine-behavior claim (classifies which trust-state PIP fields are open-core vs. Type-C; references RFC 0115 / 0117 / 0118 / 0140 §4.3.3 / §8) | n/a | Out of citation-ledger scope by design. |

| §5.1 (what PaC engines provide natively) — deterministic evaluation | Each engine's evaluation is deterministic / verification-guided | A.1 row 6; A.2 row 5 | Covered. |

| §5.1 (what PaC engines provide natively) — audit logging | Each engine exposes a documented audit-logging surface (OPA decision logs; Cedar response diagnostics; SpiceDB Watch API) | A.1 row 7; A.2 row 6; A.3 row 7 | Covered (each engine row uses the audit-logging surface that engine documents). |

| §5.2 (what PaC engines do not provide natively) | Trust scoring; privilege rings; kill switch; circuit breaker; cascade revocation — best-evidence hypothesis citing each engine's own scope statement, marked pending §10 maintainer confirmation | A.1 row 8; A.2 row 7; A.3 row 8 | Covered as best-evidence hypothesis (per the preamble's negative-claim status note). |

| §5.3 (Trust-State PIP Contract) | No engine-behavior claim (specifies the GiFo-internal PIP contract sourced from GAuth and the Gimel Auth Platform) | n/a | Out of citation-ledger scope by design. |

| §5.3.1 (Ring Model and Score-to-Tier Mapping) | No engine-behavior claim (reproduces the RFC 0140 v1.2 §7.1 governance-profile-to-ring mapping for ecosystem alignment) | n/a | Out of citation-ledger scope by design. |

| §6.1 (OPA: bundle-per-tenant) | OPA bundles can be partitioned per tenant | A.1 row 4 (re-cited as §3.1 / 6.1) | Covered. |

| §6.2 (Cedar: policy-store-per-tenant) | Cedar's per-tenant isolation via per-tenant policy store | A.2 row 5 (re-cited as §4.3 / 6.2) | Covered. |

| §6.3 (SpiceDB: relationship-namespace-per-tenant) | SpiceDB per-tenant isolation by namespacing definitions | A.3 row 8 | Covered. |

| §6.4 (GAuth credential-native multi-tenancy) | No engine-behavior claim (describes GAuth's own multi-tenancy primitive, governed by RFC 0140 and 0117) | n/a | Out of citation-ledger scope by design. |

| §6.5 (chaining pattern) | No engine-behavior claim (describes the chain itself, not engine behavior) | n/a | Out of citation-ledger scope by design. |

| §7 EC-01 (Enforcement Binding) | Phase-1 + Phase-2 hybrid is identical across CCPE-A/B; engine trust-state ownership differs | RFC 0140 §2.8 (AGT side); A.1 row 8 / A.2 row 7 / A.3 row 8 (PaC absence of native trust state) | Covered. |

| §7 EC-02 (Policy Location) | CCPE-B Phase 2 stored in Rego bundle / Cedar policy store / SpiceDB relationship database | A.1 row 4; A.2 row 5; A.3 row 8 | Covered. |

| §7 EC-03 (Granularity) | CCPE-B Phase 2 = Rego arbitrary structured / Cedar entity-typed / SpiceDB relationship-permission | A.1 row 5; A.2 row 3; A.3 row 5 | Covered. |

| §7 EC-04 (Multi-Tenant Model) | CCPE-B Phase 2 multi-tenancy = per-tenant bundle / policy store / namespace | A.1 row 4; A.2 row 5; A.3 row 8 | Covered. |

| §7 EC-05 (Multi-Hop Delegation) | PaC engines do not verify delegation hops | A.1 row 8; A.2 row 7; A.3 row 9 (best-evidence hypothesis) | Covered as best-evidence hypothesis. |

| §7 EC-06 (Human Oversight) | PaC engines do not natively offer per-action HITL | A.1 row 8; A.2 row 7; A.3 row 9 (best-evidence hypothesis) | Covered as best-evidence hypothesis. |

| §7 EC-07 (Interoperability) | Standards-based at credential layer (RFC 0116); engine policy languages governed by their own communities | RFC 0116 (credential side); A.1–A.3 row 1 each (engine community / governance frame in the engine's own self-description) | Covered. |

| §7 EC-08 (Threat Detection) | Rego/Cedar/SpiceDB do not ship a threat-detection corpus | A.1 row 8; A.2 row 7; A.3 row 9 (best-evidence hypothesis) | Covered as best-evidence hypothesis. |

| §7 EC-09 (Regulatory Alignment) | PaC engines do not generally claim regulatory mappings | A.1 row 8; A.2 row 7; A.3 row 9 (best-evidence hypothesis) | Covered as best-evidence hypothesis. |

| §7 EC-10 (Phase-2 CONSTRAIN Authority) | No engine-behavior claim (cross-spec normative property: CONSTRAIN issued exclusively by Phase 1 in both CCPE-A and CCPE-B; references RFC 0140 §2.1 and RFC 0150 §4.1) | n/a | Out of citation-ledger scope by design. |

| §8 (Security Considerations) | No engine-behavior claim (specifies GAuth-side security properties, attestation cryptographic-binding requirements citing RFC 7519 / 7800 / 8032 / 8785, and fail-closed alignment with RFC 0140 v1.2 §11.5) | A.4 rows 4, 5, 6, 8 (IETF references re-cited for §8 cryptographic-binding requirements) | Covered. |

Outstanding gaps. None for the engines and sections in scope of v0.9.5. Future revisions adding (a) per-engine behavior claims for the §A.5 adjacent engines, (b) any new engine-behavior claim introduced into §3.1 / §4.3 / §5 / §6 / §7, or (c) finer-grained sub-claims requested by an engine maintainer through §10, will introduce corresponding new rows into the appropriate subsection above. The CCPE-B-NCI / NCP / NCO non-conformant designations queued for v0.9.6 (per §5.4.1) will not require new ledger rows because they are GiFo-internal architectural classifications, not external engine-behavior claims.

Appendix B — OPA / Rego Cookbook (Informative)

This appendix shows a worked OPA / Rego integration of the CCPE-B Phase 2 evaluator. It covers (a) the trust-state PIP object shape that the Phase 2 PaC Adapter supplies to OPA on each evaluation, (b) a deny-by-default Rego policy that consumes the trust state and applies kill-switch and tier-aware logic, (c) the bundle-per-tenant deployment

pattern (RFC 0150 §6.1), and (d) the adapter wiring that maps Rego output to the Decision Envelope (RFC 0150 §4.3).

This appendix is informative; the normative integration contract is in §4–§6 of the main spec. The ring numbering, score thresholds, and `kill_switch_state` enum used below are the normative values pinned in §5.3.1 and §5.3 — this cookbook references those values rather than redefining them.

B.1 Phase 2 PaC Adapter — OPA Input Document

The adapter constructs the following input document on each evaluation, sourcing fields per §5.3:

```json
{
  "action": {
    "verb": "tool.invoke",
    "resource": "github.repository.write",
    "sensitivity": 3,
    "constraint": { "repository": "Gimel-Foundation/gifo-rfc-0150" }
  },
  "phase1": {
    "verdict": "permit",
    "mandate_ref": "urn:gauth:poa:2026:01:agent-7c4f",
    "delegation_depth": 2
  },
  "trust_state": {
    "mandate": {
      "ref": "urn:gauth:poa:2026:01:agent-7c4f",
      "governance_profile": "enterprise",
      "delegation_depth": 2,
      "budget_remaining": { "currency": "EUR", "amount": 12450.00 }
    },
```

```
    "agent": {
      "trust_score": 820,
      "privilege_ring": 2,
      "kill_switch_state": "nominal",
      "circuit_breaker_state": "closed"
    },
    "confidence_score": 0.93
  },
  "tenant": "acme-corp"
}
```

B.2 Rego Policy — Deny-by-Default with Trust-State Consumption

The following Rego module lives in a per-tenant bundle (e.g., `bundles/acme-corp/policy/ccpe.rego`). It uses the normative ring numbering and score floors from §5.3.1 and the `{nominal, tripped}` kill-switch enum from §5.3.

```rego
package ccpe.b

import future.keywords.if
import future.keywords.in

# Default deny. Every PERMIT must be explicit.
default decision := {"verdict": "deny", "obligations": {},
"diagnostics": ["no rule permitted"]}

# Top-of-evaluation kill switch. Honored regardless of any
other rule.
decision := {"verdict": "deny", "obligations": {},
"diagnostics": ["kill switch tripped"]} if {
    input.trust_state.agent.kill_switch_state == "tripped"
```

```
}

# Circuit breaker open => deny.
decision := {"verdict": "deny", "obligations": {},
"diagnostics": ["circuit breaker open"]} if {
    input.trust_state.agent.circuit_breaker_state == "open"
}

# Phase 1 must have permitted; otherwise Phase 2 cannot rescue
it.
decision := {"verdict": "deny", "obligations": {},
"diagnostics": ["phase 1 denied"]} if {
    input.phase1.verdict != "permit"
}

# Profile-aware permit. Trust score must clear the floor
required by the
# agent's governance profile (per RFC 0140 v1.2 §7.1,
reproduced in
# RFC 0150 §5.3.1), and the resource sensitivity must be
within the ring's
# reach per the CCPE-B sensitivity-ceiling rule (S <= 4 - R,
with Ring 0 =
# highest privilege reaching all sensitivities and Ring 3 =
lowest privilege
# reaching only sensitivity 1). The deployment is responsible
for placing
# the agent in a ring consistent with its profile per §5.3.1;
this policy
# enforces the score floor against the profile and the
sensitivity ceiling
# against the ring.
decision := {"verdict": "permit", "obligations": obligations,
"diagnostics": []} if {
```

```
    input.trust_state.agent.kill_switch_state == "nominal"

    input.trust_state.agent.circuit_breaker_state == "closed"

    input.phase1.verdict == "permit"

    floor :=
profile_score_floor[input.trust_state.mandate.governance_profi
le]

    input.trust_state.agent.trust_score >= floor

    input.action.sensitivity <= 4 -
input.trust_state.agent.privilege_ring

    obligations :=
obligations_by_ring[input.trust_state.agent.privilege_ring]

}


# Profile-driven trust-score floor per RFC 0140 v1.2 §7.1

# (RFC 0115 §4 governance profiles).

profile_score_floor := {

    "minimal":    500,

    "standard":   400,

    "strikt":     500,

    "enterprise": 600,

    "behoerde":   700

}


# Per-ring obligations. Ring 0 (highest privilege / most
autonomous) carries

# no extra obligations; lower-privilege rings carry tightened
concurrency,

# rate, and audit obligations. Defined for every ring (0..3)
to avoid

# unbound-variable fall-through; Ring 3 is reachable only via
explicit

# deployment downgrade per §5.3.1.

obligations_by_ring := {
```

```
    0: {},
    1: {
        "max_concurrent_invocations": 4,
        "rate_limit_per_minute": 60,
        "audit_level": "verbose"
    },
    2: {
        "max_concurrent_invocations": 2,
        "rate_limit_per_minute": 30,
        "audit_level": "verbose"
    },
    3: {
        "max_concurrent_invocations": 1,
        "rate_limit_per_minute": 10,
        "audit_level": "verbose"
    }
}
```

B.3 Bundle-per-Tenant Layout

```
bundles/
├── acme-corp/
│   ├── manifest.json          # bundle manifest, signed
│   ├── policy/
│   │   └── ccpe.rego          # the policy above
│   └── data/
│       └── tenant_config.json # tenant-specific config
consumed via data.tenant_config
├── beta-industries/
```

```
│   ├── manifest.json
│   └── ...
```

The bundle server distributes one bundle per tenant; OPA polls and atomically reloads on change. The Phase 2 PaC Adapter selects the OPA instance (or routes within a multi-tenant OPA daemon) based on the `tenant` field of the input document.

B.4 Adapter Wiring — Rego Output to Decision Envelope

```python
# Pseudocode — language-agnostic. Production code lives in the deployment's
# adapter module and is governed by the deployment's existing Rego conventions.


def evaluate_phase2_opa(action, phase1_result, trust_state, tenant):
    opa_input = {
        "action": action,
        "phase1": phase1_result,
        "trust_state": trust_state,
        "tenant": tenant,
    }
    raw = opa_client.query(
        path="data.ccpe.b.decision",
        input=opa_input,
    )
    # Normalize per RFC 0150 §4.3 invariants.
    if "verdict" not in raw or raw["verdict"] not in ("permit", "deny"):
        return DecisionEnvelope(verdict="deny",
                                obligations={},
```

```
                                diagnostics=["malformed Rego
return"])
    return DecisionEnvelope(
        verdict=raw["verdict"],
        obligations=raw.get("obligations", {}),
        diagnostics=raw.get("diagnostics", []),
    )
```

B.5 Notes for OPA Maintainers

This cookbook reflects the OPA behavior documented at the URLs cited in Appendix A.1 as of 2026-04-18. If the OPA project has updated the bundle-management surface, decision-log surface, or Rego language since that date in ways that would change the wiring above, please open an issue per §10. We will update this appendix without revising the normative spec.

Appendix C — Cedar Cookbook (Informative)

This appendix shows a worked Cedar integration of the CCPE-B Phase 2 evaluator. It covers (a) the Cedar entity / context shape that the Phase 2 PaC Adapter supplies, (b) a Cedar policy set that consumes the trust state via context attributes, (c) the policy-store-per-tenant deployment pattern (RFC 0150 §6.2), and (d) the adapter wiring that maps Cedar's Allow/Deny + diagnostics to the Decision Envelope.

This appendix is informative; the normative integration contract is in §4–§6 of the main spec. The ring numbering, score thresholds, and `kill_switch_state` enum used below are the normative values pinned in §5.3.1 and §5.3 — this cookbook references those values rather than redefining them. **Reminder (§4.3 limitation callout):** Cedar is a binary-decision engine; CCPE-B / Cedar deployments cannot carry structured Phase-2 obligations through the Decision Envelope and must rely on Phase-1 narrowing for any post-Phase-2 narrowing of agent authority.

C.1 Cedar Schema — Entity and Action Types

```cedar
namespace CCPE {
  entity Agent in [Tenant] = {
```

```
    "trust_score": Long,
    "privilege_ring": Long,
    "kill_switch_state": String,
    "circuit_breaker_state": String,
  };

  entity Tenant = {
    "id": String,
  };

  entity Resource in [Tenant] = {
    "kind": String,
    "sensitivity": Long,
  };

  action "tool.invoke" appliesTo {
    principal: [Agent],
    resource:  [Resource],
    context: {
      "phase1_verdict": String,
      "mandate_ref": String,
      "governance_profile": String,
      "delegation_depth": Long,
      "budget_remaining": Long,
      "confidence_score": Decimal,
    }
  };
}
```

C.2 Cedar Policy Set — Trust-State-Aware Permit, Kill-Switch Forbid

The following lives in a per-tenant Cedar policy store (e.g., `policy-stores/acme-corp/`). Score floors are per the normative §5.3.1 table.

```cedar
// Forbid takes precedence over Permit in Cedar.
// Kill switch tripped => unconditional forbid.
forbid (
    principal,
    action == CCPE::Action::"tool.invoke",
    resource
)
when {
    principal.kill_switch_state == "tripped"
};

// Circuit breaker open => forbid.
forbid (
    principal,
    action == CCPE::Action::"tool.invoke",
    resource
)
when {
    principal.circuit_breaker_state == "open"
};

// Phase 1 must have permitted.
forbid (
    principal,
    action == CCPE::Action::"tool.invoke",
    resource
```

```
)
when {
    context.phase1_verdict != "permit"
};


// Permit: profile-driven trust-score floor (per RFC 0140 v1.2
§7.1, reproduced
// in RFC 0150 §5.3.1) and resource-sensitivity gate (CCPE-B-
specific rule
// S <= 4 - R). The deployment is responsible for placing the
agent in a ring
// consistent with its profile per §5.3.1; this policy
enforces the score floor
// against the profile claim and the sensitivity ceiling
against the ring.
permit (
    principal,
    action == CCPE::Action::"tool.invoke",
    resource
)
when {
    principal.kill_switch_state == "nominal" &&
    principal.circuit_breaker_state == "closed" &&
    context.phase1_verdict == "permit" &&
    (
      (context.governance_profile == "minimal"    &&
principal.trust_score >= 500) ||
      (context.governance_profile == "standard"   &&
principal.trust_score >= 400) ||
      (context.governance_profile == "strikt"     &&
principal.trust_score >= 500) ||
      (context.governance_profile == "enterprise" &&
principal.trust_score >= 600) ||
```

```
      (context.governance_profile == "behoerde"   &&
principal.trust_score >= 700)
    ) &&
    resource.sensitivity <= 4 - principal.privilege_ring
};
```

C.3 Policy-Store-per-Tenant Layout

```
policy-stores/
├── acme-corp/
│   ├── schema.cedarschema      # the schema above
│   ├── policies/
│   │   ├── 001_kill_switch.cedar
│   │   ├── 002_circuit_breaker.cedar
│   │   ├── 003_phase1_gate.cedar
│   │   └── 010_ring_aware_permit.cedar
│   └── store.json              # store metadata
├── beta-industries/
│   └── ...
```

The Phase 2 PaC Adapter routes each evaluation request to the per-tenant policy store based on the tenant context.

C.4 Adapter Wiring — Cedar Decision to Decision Envelope

```python
def evaluate_phase2_cedar(action, phase1_result, trust_state,
tenant, resource):
    principal = build_agent_entity(trust_state.agent, tenant)
    resource_entity = build_resource_entity(resource, tenant)
    context = {
        "phase1_verdict": phase1_result.verdict,
        "mandate_ref": trust_state.mandate.ref,
        "governance_profile":
trust_state.mandate.governance_profile,
        "delegation_depth":
trust_state.mandate.delegation_depth,
        "budget_remaining":
trust_state.mandate.budget_remaining.amount,
        "confidence_score": trust_state.confidence_score,
    }
    cedar_decision = cedar_authorizer.is_authorized(
        principal=principal,
        action="CCPE::Action::tool.invoke",
        resource=resource_entity,
        context=context,
        policy_store=tenant_policy_store(tenant),
    )
    # Cedar returns Allow or Deny plus diagnostics. The
obligations slot is
    # left empty because Cedar is binary by design (see §4.3
limitation
    # callout and §7 EC-03). Deployments that need structured
Phase-2
    # obligations should choose an engine whose decision shape
supports them
```

```
    # (OPA / Rego), or carry constraint metadata via Cedar
policy-context

    # entity attributes the deployment commits to surfacing
through a policy

    # convention.

    verdict = "permit" if cedar_decision.decision == "Allow"
else "deny"

    return DecisionEnvelope(

        verdict=verdict,

        obligations={},

        diagnostics=[

            {"errors": cedar_decision.diagnostics.errors},

            {"satisfying_policies":
cedar_decision.diagnostics.reason},

        ],

    )
```

C.5 Notes for Cedar Maintainers

This cookbook reflects the Cedar behavior documented at the URLs cited in Appendix A.2 as of 2026-04-18. Cedar evolves rapidly; if the schema language, evaluation API, or policy-store surface has changed, please open a §10 issue. The cookbook will be updated without revising the normative spec.

Appendix D — SpiceDB Cookbook (Informative)

This appendix shows a worked SpiceDB integration of the CCPE-B Phase 2 evaluator. It covers (a) the SpiceDB schema for agent / resource / tenant relationships, (b) the trust-state injection pattern via SpiceDB caveats, (c) the relationship-namespace-per-tenant deployment pattern (RFC 0150 §6.3), and (d) the adapter wiring that maps SpiceDB's Check + Expand to the Decision Envelope.

This appendix is informative; the normative integration contract is in §4–§6 of the main spec. The ring numbering, score thresholds, and `kill_switch_state` enum used below are the normative values pinned in §5.3.1 and §5.3 — this cookbook references those values rather than redefining them.

D.1 SpiceDB Schema — Agents, Resources, Caveats

```zed
definition agent {
    relation owner: tenant
    relation in_ring: ring
}

definition ring {
    relation contains: agent
}

definition tenant {
    relation member: agent
}

definition resource {
    relation owner: tenant
    relation accessible_by: agent with trust_state_check

    permission invoke = accessible_by
}

caveat trust_state_check(
    kill_switch_state string,
    circuit_breaker_state string,
    trust_score uint,
    privilege_ring uint,
    resource_sensitivity uint,
    phase1_verdict string,
```

```
    governance_profile string
) {
    kill_switch_state == "nominal" &&
    circuit_breaker_state == "closed" &&
    phase1_verdict == "permit" &&
    // Profile-driven trust-score floor per RFC 0140 v1.2 §7.1
    // (reproduced in RFC 0150 §5.3.1).
    (
      (governance_profile == "minimal"    && trust_score >=
500) ||
      (governance_profile == "standard"   && trust_score >=
400) ||
      (governance_profile == "strikt"     && trust_score >=
500) ||
      (governance_profile == "enterprise" && trust_score >=
600) ||
      (governance_profile == "behoerde"   && trust_score >=
700)
    ) &&
    // CCPE-B sensitivity-ceiling rule (Ring 0 = highest
privilege).
    resource_sensitivity <= 4 - privilege_ring
}
```

D.2 Trust-State Injection via Caveat Context

The Phase 2 PaC Adapter supplies the trust-state object as caveat context on each `CheckPermission` call:

```python
def evaluate_phase2_spicedb(action, phase1_result,
trust_state, tenant, resource_id):
    response = spicedb_client.check_permission(
```

```
consistency=Consistency(at_least_as_fresh=current_zedtoken()),
        resource=ObjectReference(
            object_type=f"{tenant}/resource",
            object_id=resource_id,
        ),
        permission="invoke",
        subject=SubjectReference(
            object=ObjectReference(
                object_type=f"{tenant}/agent",
                object_id=trust_state.mandate.ref,
            ),
        ),
        context={
            "kill_switch_state":
trust_state.agent.kill_switch_state,
            "circuit_breaker_state":
trust_state.agent.circuit_breaker_state,
            "trust_score":
trust_state.agent.trust_score,
            "privilege_ring":
trust_state.agent.privilege_ring,
            "resource_sensitivity":
resource_sensitivity_lookup(resource_id),
            "phase1_verdict":        phase1_result.verdict,
            "governance_profile":
trust_state.mandate.governance_profile,
        },
    )
    verdict = "permit" if response.permissionship ==
HAS_PERMISSION else "deny"
    return DecisionEnvelope(
        verdict=verdict,
```

```
        obligations={},
        diagnostics=[{"checked_at":
response.checked_at_zedtoken}],
    )
```

D.3 Relationship-Namespace-per-Tenant Layout

In SpiceDB, per-tenant isolation is achieved by namespacing object types per tenant — e.g., `acme-corp/agent`, `acme-corp/resource`, `beta-industries/agent`. The schema is loaded once per tenant prefix; relationship tuples are written and read scoped to that prefix. The Phase 2 PaC Adapter constructs object references with the per-tenant prefix on every call.

A common alternative is a single shared schema with a `tenant` relation on every resource; CCPE-B accepts either pattern. The choice is a deployment decision driven by the operator's existing SpiceDB conventions.

D.4 Optional — Expand for Audit

For deployments that surface relationship-graph evidence in the unified audit trail (RFC 0140 §9), the adapter May additionally call `ExpandPermissionTree` on the same resource and permission. The resulting tree is attached to the Decision Envelope's `diagnostics` field for post-hoc audit reconstruction. Expand is comparatively expensive; deployments Should gate it on audit policy rather than calling it on every check.

D.5 Notes for SpiceDB Maintainers

This cookbook reflects the SpiceDB behavior documented at the URLs cited in Appendix A.3 as of 2026-04-18. The caveat language and zedtoken consistency model evolve; if the cookbook is out of date, please open a §10 issue. We will update this appendix without revising the normative spec.

* * *

Disclaimer: ALL DOCUMENTS AND THE INFORMATION CONTAINED THEREIN ARE PROVIDED ON AN “AS IS” BASIS AND THE CONTRIBUTOR, THE ORGANIZATION THEY REPRESENT OR ARE SPONSORED BY (IF ANY), THE GIMEL FOUNDATION, AND ANY APPLICABLE MANAGERS OF ALTERNATE DOCUMENT STREAMS, DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION THEREIN WILL NOT INFRINGE ANY RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

PRODUCT NAMES, TRADEMARKS, AND REGISTERED TRADEMARKS REFERENCED IN THIS DOCUMENT ARE THE PROPERTY OF THEIR RESPECTIVE OWNERS AND ARE USED FOR IDENTIFICATION PURPOSES ONLY. NO ENDORSEMENT IS IMPLIED.

* * *